BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION [Secție]

DIPLOMA THESIS

[Titlu lucrare]

Supervisor
[Grad, titlu și nume coordonator]

Author
[Nume student]

2026

ABSTRACT

Abstract: un rezumat în limba engleză cu prezentarea, pe scurt, a conținutului pe capitole, punând accent pe contribuțiile proprii și originalitate

DECLARATION OF GENERATIVE AI AND AI-ASSISTED TECHNOLOGIES IN THE WRITING PROCESS

During the preparation of this work the author used Claude Opus 4.6 in order to help clarify concepts and structure paragraphs more effectively. After using this tool/service, the author reviewed and edited the content as needed and takes full responsibility for the content of the thesis.

Contents

Chapter 1

Introduction

Introducere: obiectivele lucrării și descrierea succintă a capitolelor, prezentarea temei, prezentarea contribuției proprii, respectiv a rezultatelor originale și menționarea (dacă este cazul) a sesiunii de comunicări unde a fost prezentată sau a revistei unde a fost publicată.

Chapter 2

State of the Art

This chapter outlines the theoretical foundation and current research landscape for the detection of architectural anti-patterns in microservice-based systems.

2.1 Microservices Architecture

A collection of loosely coupled services that can be independently deployed, each implementing a specific business requirement, are hereinafter referred to as microservices [**?**]. The aforementioned architectural style allows engineers to develop, deploy and scale individual services independently, unlike monolithic architectures, where the entire application is built, tested and deployed as a single unit [**?**].

Several key principles underpin the microservices architectural style. First, each service should embody a single responsibility, encapsulating a single business domain in accordance with Domain-Driven Design (DDD) [**?**]. Closely related is the principle of independent deployability, whereby services can be released without requiring coordinated deployments across the entire system. Furthermore, microservices, advocated for decentralized data management, in which each service owns its data and exposes it only through well-defined APIs, following the database-per-service pattern [**?**]. Communication between services relies on lightweight protocols such as HTTP/REST or asynchronous messaging, avoiding heavyweight middleware. Lastly, fault isolation ensures that failures in one service do not cascade to others, a goal commonly achieved through patterns such as circuit breakers or bulkheads [**?**].

Figure 2.1 demonstrates the fundamental structural distinctions between a monolithic architecture and a microservices architecture. In the monolithic approach, all components reside within a single deployable unit sharing the same database. Conversely, in the microservices approach, each service is independently deployed and manages its own database.

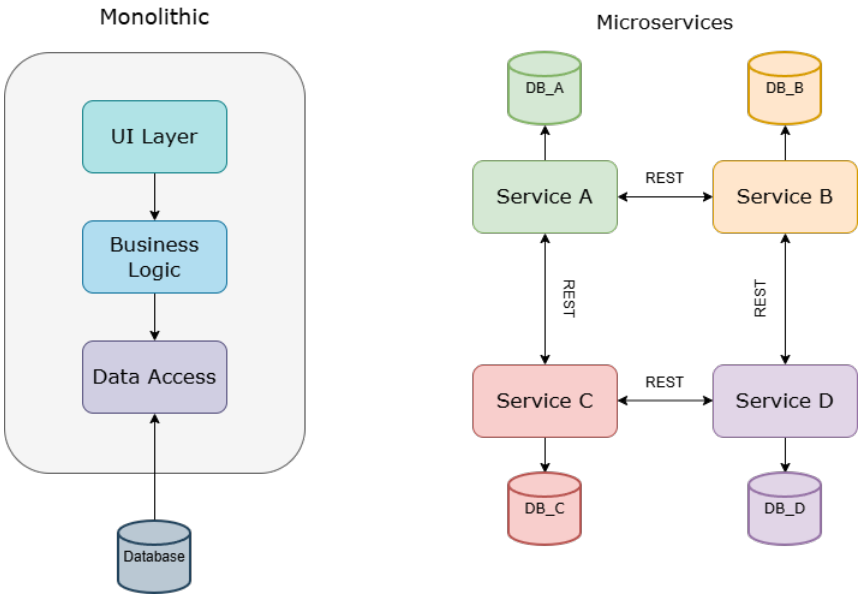


Figure 2.1: Comparison of monolithic architecture (left) and microservices architecture (right). In the monolithic approach, all modules share a singled process and database. In the microservices approach, each service is independently deployable with its own database.

While microservices offer benefits in scalability, team autonomy and technology heterogeneity, they also introduce significant overhead in terms of distributed systems challenges, e.g. network latency, partial failures, data consistency [**?**]. When these challenges are poorly managed, they manifest as architectural anti-patterns, i.e. recurring design decisions that negatively impact the quality attributes of the system.

2.2 Microservices Anti-Patterns

An anti-pattern denotes a frequently used solution to a recurring problem that, rather than resolving it efficiently, introduces a set of negative consequences that may exacerbate the issue further [**?**]. In the context of microservices, anti-patterns represent architectural or design decisions that undermine the benefits of the microservices pattern. This section defines and characterizes the anti-patterns targeted by this dissertation.

2.2.1 Shared Database

The Shared Database anti-pattern occurs when multiple microservices are allowed access to modify the same database schema, which is a violation of the principle of decentralized data management [**?**]. This creates tight coupling between

services at the data layer, as changes to the database schema can break multiple services simultaneously. Furthermore, services cannot be independently deployed or scaled without considering the shared database.

As illustrated in Figure 2.2, when services A, B and C all connect to the same database, any schema migration in one service can cause failures in the other. The recommended remediation is to adopt the database-per-service pattern and synchronize data across services using asynchronous events or the Saga pattern [**?**].

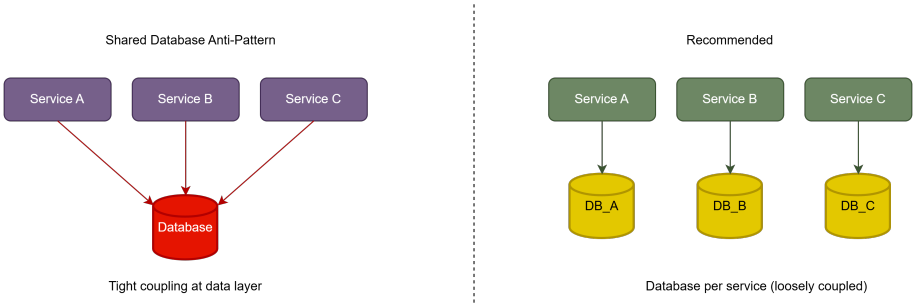


Figure 2.2: Shared Database Anti-Pattern: multiple services access the same database instance, creating tight coupling at the data layer.

Detection of this anti-pattern can be performed through static analysis of service configuration files. Particularly, in Java Spring Boot applications, the `application.yml` or `application.properties` files contain datasource connection URLs (e.g. `spring.datasource.url`). By comparing these URLs across all detected microservices within a project, instances of shared databases can be identified automatically.

2.2.2 Cyclic Dependency

Cyclic dependencies occur when two or more services form a circular chain of dependencies (e.g., in an online shopping application, the Order Service depends on the Payment Service, which depends on the Inventory Service, which in turn depends back on the Order Service, forming a circular dependency). Cycles in the service dependency graph violate the acyclic dependencies principle and create tight coupling, making it fairly difficult to deploy, test, or reason about services independently [**?**].

Cycle detection in directed graphs is a well-studied problem in computer science. A common approach used when endeavouring to find strongly connected components (SCCs) is Tarjan's algorithm, which operates in $O(V + E)$ time, where V is the number of vertices, i.e. services, and E is the number of edges, i.e. dependencies [**?**]. Any strongly connected component with more than one vertex represents a cycle in the dependency graph.

The dependency graph itself is constructed by analyzing inter-service communication patterns. In Java-based microservices, dependencies can be identified through static analysis of:

- `@FeignClient` annotations (Spring Cloud OpenFeign)
- `RestTemplate` usage patterns
- `WebClient` invocations (Spring WebFlux)
- gRPC client stubs
- Message queue producers and consumers

Figure 2.3 illustrates a cyclic dependency among three services, as presented in the introduction of this subsection.

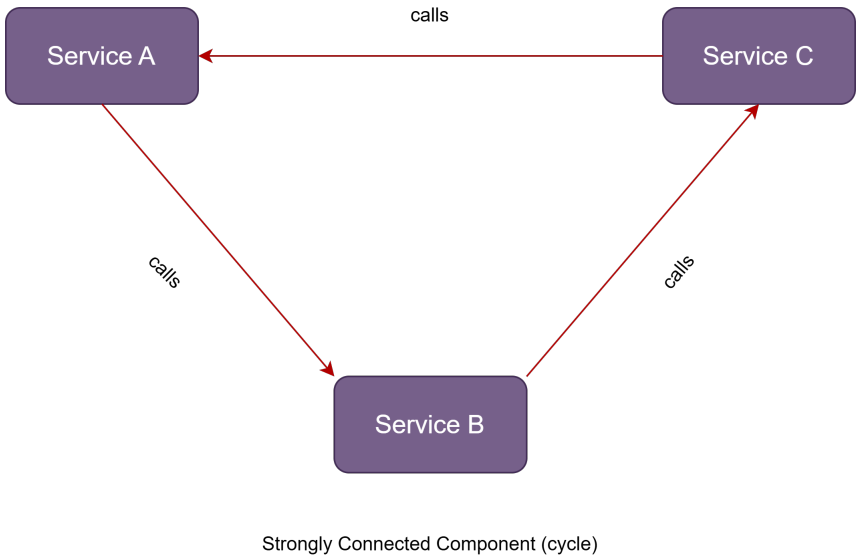


Figure 2.3: Example of a cyclic dependency. Services A, B and C forma a strongly connected component where no service can be deployed or tested independently.

2.2.3 Nano Service

A service that is too fine-grained to justify its operational overhead, is hereinafter named "Nano Service" [**?**]. Each microservice carries inherent costs: monitoring, deployment pipelines, network communication, and operational maintenance. When a service has very few lines of code and exposes only one or two endpoints, the overhead of running it as an independent process may outweigh the benefits of decomposition.

The detection of nano services requires defining quantitative thresholds. In this work, a service is classified as a nano service when it satisfies the conjunction of two conditions:

$$\text{LOC}(s) < \theta_{\text{LOC}} \wedge \text{endpoints}(s) \leq \theta_{\text{endpoints}} \tag{2.1}$$

where θ_{LOC} is the maximum lines-of-code threshold (default: 500) and $\theta_{\text{endpoints}}$ is the maximum endpoint count threshold (default: 2). These thresholds are configurable and should be calibrated based on the specific project context.

The recommended remediation for nano services is to merge them with a related, functionally cohesive service [**?**].

2.2.4 God Service

A God Service is the microservice counterpart of the well-known God Class code smell [**?**]. It refers to a service that handles too many responsibilities, spanning multiple business domains within a single deployable unit. God services grow over time as developers add functionality to an existing service rather than creating new ones, resulting in a service that is difficult to understand, test, and scale independently.

Detection of god services in this work is based on the presence of code-level indicators. Specifically, a microservice is flagged as a god service when it contains a number of God Class code smells (as detected by DesigniteJava) exceeding a configurable threshold:

$$|\{c \in \text{classes}(s) : \text{isGodClass}(c)\}| \geq \theta_{\text{domains}} \tag{2.2}$$

where θ_{domains} is the minimum number of God Class occurrences that indicates excessive responsibility concentration (default: 3).

2.2.5 Chatty Service

The Chatty Service anti-pattern occurs when a service makes an excessive number of fine-grained remote calls to another service to complete a single business operation [**?**]. This increases network latency, reduces throughput, and makes the system fragile to network failures. The chatty communication pattern often indicates that the service boundaries were drawn incorrectly, and data that should be co-located is split across services.

Detection involves analyzing the dependency graph and identifying edges with a call count exceeding a configurable threshold:

$$\text{callCount}(s_1, s_2) \geq \theta_{\text{chatty}} \tag{2.3}$$

where θ_{chatty} is the minimum call count threshold (default: 5).

Figure 2.4 contrasts a chatty communication pattern with a well-designed coarse-grained interaction. In the chatty pattern, Service A makes many fine-grained calls to Service B to complete a single operation, whereas the improved design consolidates these into fewer, coarser-grained requests.

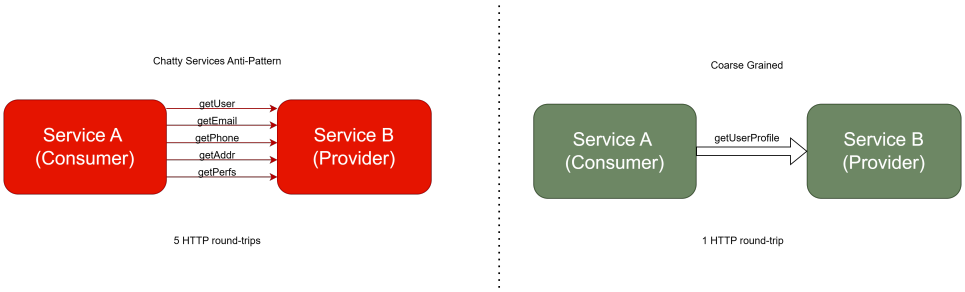


Figure 2.4: Example of chatty services versus coarse-grained services. Chatty Service A sends too many requests to Service B, whereas Coarse-Grained Service A sends fewer, more consolidated requests.

2.2.6 Hardcoded Endpoints

The Hardcoded Endpoints anti-pattern occurs when service URLs are embedded directly in the source code instead of being resolved through a service discovery mechanism (e.g., Netflix Eureka, Consul, or Kubernetes DNS) [**?**]. Hardcoded endpoints create tight coupling between services and make the system inflexible to infrastructure changes such as scaling, failover, or environment migration.

Detection involves scanning the source code and configuration files for patterns such as `http://`, `https://`, `localhost:`, or explicit IP addresses in REST client configurations.

2.2.7 Distributed Monolith

The Distributed Monolith anti-pattern describes a system that has been decomposed into multiple services but retains the tight coupling characteristics of a monolithic architecture [**?**]. In such a system, services cannot be deployed, tested, or scaled independently — a change in one service frequently requires coordinated changes and redeployments across several other services. The result is a system that combines the complexity of distributed systems (network latency, partial failures, operational overhead) with none of the benefits of microservices (independent deployability, team autonomy, technology heterogeneity).

A distributed monolith typically arises when services share too many dependencies, communicate excessively, or rely on synchronous call chains to fulfill requests.

The coupling coefficient C (Equation 2.4) serves as a quantitative indicator: when a large proportion of all possible inter-service dependencies actually exist, the system behaves more like a monolith rather than a suite of decoupled services, each maintaining a high degree of operational independence. In this work, a system is flagged as a potential distributed monolith when the coupling coefficient exceeds a configurable threshold:

$$C = \frac{|E|}{|V| \times (|V| - 1)} > \theta_{\text{monolith}} \tag{2.4}$$

where θ_{monolith} is the coupling coefficient threshold above which the system is considered excessively coupled.

The recommended remediation involves identifying and removing unnecessary inter-service dependencies, introducing asynchronous communication where possible, and ensuring that each service encapsulates a well-defined bounded context with minimal external dependencies [**?**].

2.2.8 Additional Anti-Patterns

Beyond the aforementioned primary anti-patterns, this work also considers API Versioning Absence.

API Versioning Absence refers to a harmful practice when services expose APIs without a versioning strategy, thus making backward-compatible evolution impossible without breaking consumers.

Table 2.1 summarizes all the anti-patterns targeted by this dissertation, along with their severity levels and detection approaches.

2.3 Tools and Technologies

This section provides an overview of the tools and technologies used to support the development of the anti-pattern detection system presented in this dissertation.

2.3.1 Spring Boot

Spring Boot is a widely adopted framework for developing enterprise-grade Java applications requiring minimal setup overhead [1]. It offers auto-configuration, integrated serve support, and a comprehensive suite of supporting tools and libraries for building web applications, RESTful APIs, and data access layers. In this work, Spring Boot 4.0 serves as the application framework, providing:

[1] Spring Boot

Anti-Pattern	Severity	Detection Approach
Shared Database	High	Comparison of datasource URLs across services
Cyclic Dependency	Critical	Tarjan's SCC algorithm on the dependency graph
Nano Service	Medium	LOC and endpoint count thresholds
God Service	High	God Class code smell frequency via DesigniteJava
Chatty Service	High	Call count analysis on dependency edges
Hardcoded Endpoints	Medium	Pattern matching in source and configuration files
Distributed Monolith	Critical	Coupling coefficient analysis
API Versioning Absence	Medium	Endpoint path analysis for version indicators

Table 2.1: Summary of targeted microservice anti-patterns.

- **Spring Web:** RESTful API endpoints for project upload - either through a zip file or GitHub GitLab source code URL, analysis job management and result retrieval.
- **Spring Data JPA:** Object-relational mapping and repository abstractions for PostgreSQL persistence.
- **Spring Security:** JWT-based authentication and authorization.
- **Spring Async:** Asynchronous task execution for long-running analysis jobs.

2.3.2 PostgreSQL

PostgreSQL is an open-source relational database management system widely recognised for its reliability, comprehensive functionality and architectural extensibility [2]. It is used in this disseration to persist metadata, microservice information, code smell reports, code snippets, detected anti-patterns, and analysis results. The database schema supports queries for aggregation and reporting, such as counting anti-patterns by severity or computing per-service code smell frequencies.

2.3.3 Spoon

Spoon is used for Java source code analysis, as it provides a complete metamodel

[2]PostgreSQL

of the Java language and supports visitor-based and query-traversal of the Abstract Syntax Tree [**?**].

2.3.4 JGit

JGit is a pure Java implementation of the Git version control system [3]. It is used in this work to clone Git repositories submitted for analysis, enabling users to provide a repository URL rather than uploading source archives manually.

2.3.5 DesigniteJava

DesigniteJava is a command-line tool that performs design smell detection on Java projects [**?**]. It accepts a source directory as input and produces CSV files containing detected smells. In this dissertation, it is invoked as a Java subprocess with configurable timeout settings.

2.3.6 Docker

Docker is used for containerized deployment of the application and its dependencies. Both the Backend and the Frontend projects in this dissertation define a `docker-compose.yml` file, which outlines the multi-container setup including the Spring Boot application and PostgreSQL database.

2.4 Related Work

The detection of anti-patterns in microservices architectures has received growing attention from the scientific research community. This section reviews the most relevant prior work and positions this dissertation within the existing landscape.

2.4.1 Systematic Reviews and Grey Literature Studies

Several systematic literature reviews have mapped the state of research on microservices, providing a broad understanding of the field's maturity and open challenges.

Di-Francesco et al. conducted a systematic mapping study on research related to microservices architecture, analyzing 71 studies published between 2014 and 2016 [**?**]. Their findings revealed that the majority of research focused on migration from monolithic to microservice architectures, with comparatively little attention to quality assurance, anti-pattern detection, and automated analysis tools. The authors

[3] JGit

identified an appreciable gap between the industrial adoption of microservices and the availability of rigorous academic research that addresses the architectural challenges that practitioners face. This gap motivates the need for tools that can aid development teams in identifying architectural debt in microservice systems.

Soldani et al. performed a systematic grey literature review covering 51 industrial experience reports and practitioner blog posts on microservices [**?**]. Their study identified the most frequently reported pains (challenges) and gains (benefits) of microservice adoption. Among the pains, the most commonly reported were increased complexity in distributed systems, difficulties with data management acorss services, and challenges related to monitoring and debugging. Notably, several of the anti-patterns targeted by this dissertation correspond to the pains reported by the practitioners in the aforementioned study: the Shared Database anti-pattern relates to data management difficulties, the Distributed Monolith corresponds to accidental tight coupling, and the Chatty Service pattern reflects the complexity of managing inter-service communication. The alignment between practitioner-reported challenges and the anti-patterns detected by this work supports its relevance.

2.4.2 Empirical Studies on Microservices Anti-Patterns

Taibi and Lenarduzzi conducted an empirical study involving interviews with 72 developers from 61 companies, identifying the most common ”bad smells” in microservice architectures [**?**]. Their findings highlighted that Wrong Cuts, Shared Database (also referred to as ”Shared Persistency”) and Hardcoded Endpoints are among the most frequently encountered and impactful anti-patterns in enterprise practice. The study also reported that many teams were unaware of these anti-patterns until they had accumulated enough significant technical debt, underscoring the value of automated detection tools. This empirical evidence informed the selection of anti-patterns targeted by this dissertation.

In a subsequent work, Taibi, Lenarduzzi, and Pahl proposed a structured taxonomy of microservice anti-patterns [**?**]. They classified anti-patterns along two dimensions: the architectural level at which they manifest (e.g. service design, communication, data management, deployment) and their root cause (e.g. improper decomposition, misused patterns, technology misue). This taxonomy provides a principled framework for organizing detection efforts. The anti-patterns addressed in this dissertation span all four architectural levels, as summarized in Table

Bogner et al. conducted a systematic literature review investigating the assessment of maintainability in service- and microservice based systems [**?**]. They surveyed a handful of approaches to measuring quality attributes related to maintainability in service-oriented architectures and found that most existing metrics focus

Architectural Dimension	Anti-Pattern	Concern Addressed
Service Design	Nano Service	Service too small to justify operational overhead
	God Service	Service handling too many responsibilities
Communication	Chatty Service	Excessive fine-grained inter-service calls
	Cyclic Dependency	Circular dependency chains between services
	Hardcoded Endpoints	Service URLs hardcoded instead of using discovery
Data Management	Shared Database	Multiple services accessing the same database
Deployment & Coupling	Distributed Monolith	Tightly coupled services requiring co-deployment
	API Versioning Absence	No API versioning strategy detected

Table 2.2: Mapping of the eight anti-patterns detected by this work to the architectural dimensions defined by Taibi et al. [**?**].

on interface complexity and coupling. However, few approaches provide automated tool support or integrate code-level analysis with architectural-level metrics. This observation aligns with a notable contribution of this dissertation: combining intra-service code smell detection with inter-service dependency analysis to provide a panoramic assessment of architectural quality.

2.4.3 Static Analysis Approaches

Neri et al. introduced an index of design principles for microservices and developed static analysis techniques to detect violations [**?**]. Their work formalized seven design principles (including service autonomy, decentralized data management, and smart endpoints/dumb pipes) and defined 16 architectural smells as violations of these principles. Their approach focuses on detecting violations through analysis of service interfaces, deployment descriptors, and data models. While their work provides a strong principled foundation for reasoning about microservice quality, it does not integrate code-level smell detection into the architectural analysis and does not provide an implemented tool with configurable thresholds.

Sharma and Spinellis, the creators of DesigniteJava, provided a comprehensive survey on software smells and demonstrated the role of automated tools in detecting them [**?**]. DesigniteJava detects design smells (e.g. God Class, Feature Envy), implementation smells (e.g. Long Method, Complex Method), and architecture smells (e.g. Cyclic Dependency, Hub-like Dependency). While DesigniteJava operates at

code level within individual projects, this dissertation uses its output as input to higher-level architectural analysis, connecting code smells and microservice anti-patterns. Specifically, God Class smell frequency is used as a signal for god service detection, and Feature Envy is used as an indicator of misplaced service boundaries (Wrong Cuts).

Pigazzini et al. proposed an approach for detecting microservice smells by combining structural analysis with dependency extraction [**?**]. Their work focused on identifying smells such as Cyclic Dependencies, Hardcoded Endpoints, and Shared Persistence by analyzing dependency structure extracted from build files and Docker configurations. While their approach addresses inter-service structural smells, it does not incorporate intra-service code smell analysis and does not support the breadth of anti-patterns covered by this dissertation.

Cerny et al. provided a contextual analysis of microservice architecture, surveying current research directions and identifying gaps in automated analysis capabilities [**?**]. They highlighted the need for tools that can automatically reconstruct microservice architectures from source code and detect anti-patterns without requiring manual configuration. The automated microservice detection mechanism implemented in this work directly addresses this need by scanning for build files and source directories to identify service boundaries.

2.4.4 Dynamic and Hybrid Approaches

Dynamic analysis approaches monitor running systems to detect anti-patterns from runtime behaviour. Distributed tracing tools such as Jaeger or Zipkin can reveal chatty communication patterns and cyclic call chains at runtime by recording the flow of requests across service boundaries. However, dynamic approaches require a running system with representative workloads and instrumented services, making them less suitable for early-stage analysis, code review workflows, or integration into CI/CD pipelines where the system is not deployed.

Hybrid approaches combine static and dynamic analysis. For instance, a tool might perform static dependency extraction from source code and complement it with runtime call frequency data collected via distributed tracing. While hybrid approaches can provide higher accuracy (e.g. by confirming tha a statically detected dependency is actually exercised at runtime), they introduce additional complexity and infrastructure requirements. They also require the system to be deployed and exercised with representative traffic, which is not always feasible.

This dissertation focuses on static analysis to enable anti-pattern detection without requiring a running system, making it applicable to source repositories at any stage of development. This design choice prioritizes broad applicabiltiy and ease of

integration over the higher precision that runtime data could provide.

2.4.5 Automated Detection Tools

Several tools have been developed for microservice analysis. This subsection reviews the most relevant ones and highlights their strengths and limitations relative to this work.

Arcan serves as a tool developed for detection and analysis of architectural smells in Java projects that supports dependency graph analysis and cycle detection [**?**]. It detects architectural smells such as Cyclic Dependencies and Unstable Dependencies by constructing and analyzing a dependency graph from compiled Java bytecode. However, Arcan was primarily designed with monolithic Java applications in mind and does not specifically target microservice anti-patterns such as Shared Database (or Persistence), Chatty Service or Distributed Monolith. It also does not provide automated microservice boundary detection or a composite health score.

MicroART reconstructs microservice architecures from source code and Docker configurations, providing visualizations capabilities [**?**]. It parses `Dockerfile` and `docker-compose.yml` files to identify services and their relationships, and it generates graphical representations of the reconstructed architecture. While MicroART does address the important problem of architecture recovery, it focuses on visualization rather than anti-pattern detection. It does not analyze source code for code smells, does not compute coupling metrics and does not provide remediation recommendations.

MSANose is a detection tool specifically designed for microservice smells, based on the catalog established by Taibi et al. [**?**]. It detects smells such as Shared Database, Cyclic Dependencies, and Hardcoded Endpoints through static analysis for Spring Boot projects. MSANose represents the closest prior work to this dissertation. However, it differs in several respects: it does not integrate code-levle smell detection (e.g. God Class analysis), it does not compute a composite health score, it does not support configurable detection thresholds and it does not detect anti-patterns such as API Versioning Absence or God Service.

Table 2.3 provides a detailed feature comparison of these tools against the system presented in this dissertation.

2.4.6 Positioning and Contributions of This Work

This dissertation differentiates itself from existing work in several ways, addressing gaps identified in the reviewed literature.

First, this work performs multi-level analysis. Existing tools typically operate at a single level of abstraction: either code-level smell detection (e.g. Designite-

Feature	Arcan	MicroART	MSANose	This Work
Targets microservices	×	✓	✓	✓
Code smell detection	✓	×	×	✓
Cycle detection	✓	×	✓	✓
Shared database detection	×	×	✓	✓
God service detection	×	×	×	✓
Chatty service detection	×	×	×	✓
Hardcoded endpoints	×	×	✓	✓
API versioning check	×	×	×	✓
Auto service discovery	×	Partial	×	✓
Health score metric	×	×	×	✓
Web interface / API	×	✓	×	✓
Configurable thresholds	×	×	×	✓
Code snippet evidence	×	×	×	✓
Remediation guidance	×	×	×	✓
Target language	Java	Multi	Java	Java
Analysis technique	Static	Static	Static	Static

Table 2.3: Feature comparison of related architectural analysis tools. ✓ indicates the feature is supported, × indicates it is not supported. "Partial" indicates limited support.

Java) or architectural-level dependency analysis (e.g. Arcan, MicroART). This work combines intra-service code smell detection (via DesigniteJava) with inter-service architectural analysis (via Spoon-based Abstract Syntax Tree analysis and graph algorithms), providing a more comprehensive assessment. For example, God Class smells detected at the code level are used as evidence for flagging god services at the architectural level.

Second, the tool offers broader anti-pattern coverage, detecting nine distinct anti-patterns across four architectural dimensions: service design (Nano Service, God Service), inter-service communication (Chatty Service, Cyclic Dependency, Hard-coded Endpoints), data management (Shared Database), and system-level coupling (Distributed Monolith, API Versioning Absence). This represents a broader coverage than any single existing tool reviewed in this chapter.

Third, the tool provides automated microservice detection by automatically identifying microservice boundaries within a multi-module project through scanning for build files (`pom.xml`, `build.gradle`, `build.gradle.kts`) and verifying the presence of source directories. This eliminates the need for manual service enumeration, which is required by most existing tools. Multi-module Maven and Gradle projects are supported, with automatic filtering of parent aggregator modules.

Fourth, detection thresholds for metrics-based anti-patterns are externalized as configurable detection parameters (via Spring Boot's `application.yml`), allowing calibration to project-specific norms. For instance, the LOC threshold for nano service detection or the call count threshold for chatty service detection can be adjusted without modifying the source code.

Fifth, a composite health score provides a single numeric metric (0–100) for architectural quality, accompanied by a letter grade (A through F). Unlike a simple aggregate, the score is decomposed into four weighted categories — anti-pattern severity, code quality, architectural coupling, and service sizing — each with an itemized list of deductions. This category-based breakdown enables teams to identify *which* quality dimension is degraded, track architectural health trends over successive analysis runs, and measure the concrete impact of refactoring efforts.

Sixth, unlike existing tools that report anti-patterns as abstract findings, this work supports evidence-based reporting with code snippets attached to each detected anti-pattern. When a cyclic dependency is detected, the tool extracts and presents the relevant `@FeignClient` annotation or `RestTemplate` invocation from the source code, providing developers with immediate context for understanding and remediating the issue.

Finally, the tool is delivered as a web application with a RESTful API, supporting both interactive use through a web dashboard and programmatic integration into CI/CD pipelines for continuous architectural quality monitoring. Users can

submit projects for analysis either by uploading a ZIP archive or by providing a Git repository URL, which is cloned automatically via JGit.

2.5 Summary

This chapter has established the theoretical and technological foundation for this dissertation. The microservices architectural style was introduced along its core principles, inherent challenges and a taxonomy of eight microservice anti-patterns. The tools and technologies ussed in the implementation were surveyed as well.

The related work was reviewed across four categories: systematic literature reviews that map the research landscape, empirical studies that identify practitioner-reported anti-patterns, static analysis approaches that formalize detection techniques, and automated tools that implement detection. The review revealed that existing tools tend to operate at a single level of abstraction and cover a limited set of anti-patterns. This dissertation addresses these gaps by combining intra-service and inter-service analysis, supporting nine anti-patterns across four architectural dimensions, providing automated microservice detection, configurable thresholds, evidence-based reporting with code snippets, and a category-based composite health score with letter grading for tracking architectural quality over time. The next chapter presents the detailed detection methodology for each anti-pattern.

Chapter 3

Detection Methodology

This chapter presents the detection methodology employed by the system developed in this dissertation. It begins with an overview of the analysis pipeline and the foundational techniques, i.e. static code analysis, AST-based source code inspection and code smell detection, then describes the construction and analysis of the inter-service dependency graph and the composite health score computation. The chapter concludes with a detailed description algorithm for each individual anti-pattern.

3.1 Detection Techniques

This section outlines the techniques used for detecting architectural anti-patterns in microservice systems. Figure 3.1 provides an overview of the analysis pipeline, showing how source code is processed through multiple stages to produce anti-pattern detection results.

3.1.1 Static Code Analysis

Static code analysis examines source code without executing it, allowing detection of structural issues, code smells, and architectural violations at the source level

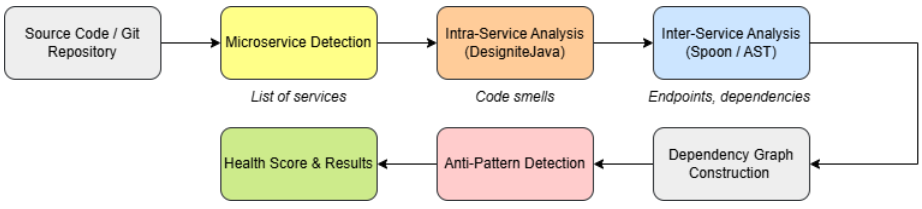


Figure 3.1: Overview of the anti-pattern detection pipeline. Source code is processed through microservice detection, intra-service and inter-service analysis, dependency graph construction, and anti-pattern detection, producing a health score and detailed results.

[**?**]. In the context of microservices, static analysis can be applied at two levels. Intra-service analysis examines the internal structure of each microservice for code-level smells such as God Class, Long Method, Feature Envy, and Complex Method; these smells serve as indicators of architectural problems at the service level. Inter-service analysis, on the other hand, examines the communication patterns, dependencies, and shared resources across microservices to detect architectural anti-patterns such as cyclic dependencies and shared databases.

3.1.2 Abstract Syntax Tree Analysis with Spoon

Spoon is an open-source library for analyzing, transforming and generating Java source code [**?**]. It builds a fine-grained Abstract Syntax Tree (AST) that provides full access to all elements of the Java programming language, including annotations, method invocations and type references.

In this work, Spoon is used for inter-service analysis, specifically in three ways. Firstly, it detects REST controller endpoints by scanning for `@RestController`, `@GetMapping`, `@PostMapping` and other related annotations. Furthermore, it is used to identify inter-service communication by analyzing `@FeignClient` annotations, `@RestTemplate` method calls, and `@WebClient` builder patterns. Lastly, it extracts service metadata such as the number of endpoints, controller classes and API versioning information.

The AST-based approach is more robust than regular expression-based scanning, as it understands the syntactic structure of the code and can resolve annotation attributes, method call chains and type hierarchies.

3.1.3 Code Smell Detection with DesigniteJava

DesigniteJava is a static analysis tool that detects design smells, implementation smells and architecture smells in Java projects [**?**]. It analyzes Java source code and produces CSV reports categorizing detected smells by type, affected class and file location.

The tool detects three categories of smells, summarized in Table 3.1.

In this dissertation, DesigniteJava is integrated as an external tool invoked as a subprocess. The analysis pipeline passes each detected microservice's source directory to DesigniteJava, parses the resulting CSV output files (`DesignSmells.csv`, `ImplementationSmells.csv`, `ArchitectureSmells.csv`), and stores the results in the application database. The God Class smell frequency per service is then used as a signal for god service detection (see Section 2.2.4).

Category	Smell Name	Description	Severity
Design	God Class	Class with too many responsibilities	High
	Feature Envy	Method more interested in other classes	High
	Insufficient Modularization	Large, poorly modularized class	Medium
	Unnecessary Abstraction	Abstraction without clear purpose	Low
Implementation	Long Method	Method with excessive length	Medium
	Complex Method	Method with high cyclomatic complexity	Medium
	Long Parameter List	Method with too many parameters	Medium
	Magic Number	Unexplained numeric literals	Low
Architecture	Cyclic Dependency	Circular dependencies between packages	High
	Unstable Dependency	Depending on less stable components	Medium
	Hub-like Dependency	Class depended upon by many others	Medium

Table 3.1: Code smell categories detected by DesigniteJava, with their descriptions and severity mappings used in this work.

3.1.4 Dependency Graph Construction and Analysis

The dependency graph is a directed graph $G = (V, E)$ where each vertex $v \in V$ represents a microservice and each edge $e = (s, t) \in E$ represents a dependency from service s to service t. Edges are annotated with metadata including the dependency type (REST synchronous, Feign client, messaging, etc.) and the call count.

A central challenge in dependency graph construction is resolving the *target* of an inter-service call to an actual microservice in the project. In real-world codebases, services are referenced by a variety of identifiers: directory names, `spring.application.name` values, `localhost:PORT` URLs, or Spring property placeholders injected via `@Value` annotations. To address this, the dependency graph builder employs a multi-strategy resolution pipeline.

The first step is a configuration pre-scan: before scanning source code, each service's `application.yml` or `application.properties` is parsed. The `spring.application.name` property is registered as an alias for the service directory name, and `server.port` values are recorded in a port-to-service mapping. Next, the pipeline performs `@Value` field resolution, in which fields annotated with Spring's `@Value` (e.g., `@Value("${order-service.url:http://localhost:8081}")`) are pre-scanned using Spoon. Property placeholders are resolved against the service's configuration, falling back to default values when the property key is not found. Finally, a cascading target resolution strategy is applied: when a call target is extracted from a `@FeignClient` annotation, `RestTemplate` invocation, or `WebClient` call, it is matched to a microservice using three strategies applied in order — exact match against service names and registered aliases, port-based match-

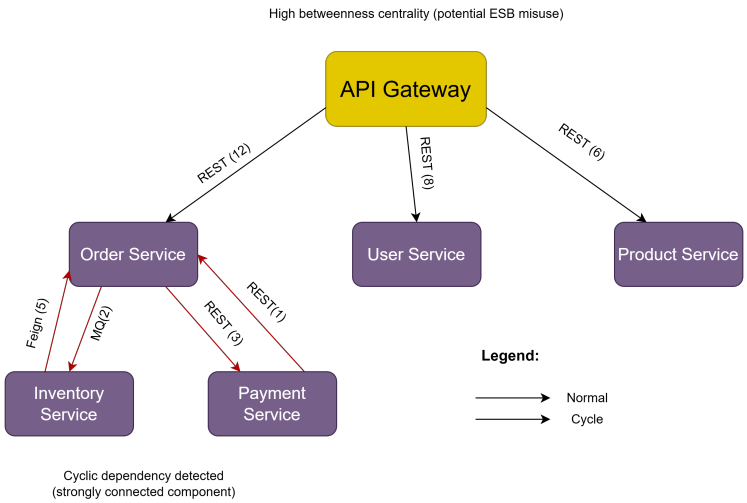


Figure 3.2: Example dependency graph for a microservice system. Nodes represent services, edges represent dependencies with type and call count annotations. The red dashed edges indicate a cyclic dependency. The API Gateway node exhibits high centrality, potentially indicating ESB misuse.

ing for `localhost:PORT`-style targets, and fuzzy substring matching with common suffix stripping (e.g., matching "order" to "order-service").

Figure 3.2 shows an example dependency graph for a hypothetical microservice system. Each node represents a service, and edges are labeled with the dependency type and call count. The graph reveals a cycle between the Order, Inventory and Payment services as well as a potential ESB misuse pattern where the API Gateway mediates most communication.

Graph-based anti-pattern detection encompasses several analytical techniques. Cycle detection involves finding strongly connected components using Tarjan's algorithm to identify cyclic dependencies, while centrality analysis computes betweenness centrality to detect potential ESB misuse, where a single service mediates a disproportionate share of communication. Additionally, coupling metrics are evaluated by computing the coupling coefficient as the ratio of actual dependencies to possible dependencies.

$$C = \frac{|E|}{|V| \times (|V| - 1)} \tag{3.1}$$

A high coupling coefficient suggests that the system may be a distributed monolith rather than a collection of loosely coupled services.

3.1.5 Health Score Computation

To provide an overall assessment of the system's architectural health, this dissertation computes a composite health score based on the detected anti-patterns and

their severity levels. The score starts at 100 and is decremented based on the number and severity of detected issues:

$$H = \max\Big(0,\ 100 - 15\,n_{\text{crit}} - 10\,n_{\text{high}} - 5\,n_{\text{med}} - 2\,n_{\text{low}}\Big) \tag{3.2}$$

where n_{crit}, n_{high}, n_{med}, and n_{low} represent the number of detected anti-patterns at each severity level. This scoring mechanism provides a quick assesment that can be tracked over time to monitor architectural health trends.

Table 3.2 details the severity levels, their corresponding penalty weights in the health score formula, and examples of anti-patterns assigned to each level.

Severity	Weight	Penalty	Example Anti-Patterns
Critical	4	-15 per issue	Cyclic Dependency, Distributed Monolith
High	3	-10 per issue	Shared Database, God Service, Wrong Cuts, Chatty Service, ESB Misuse
Medium	2	-5 per issue	Nano Service, Hardcoded Endpoints, API Versioning Absence
Low	1	-2 per issue	—

Table 3.2: Severity levels with their weights and penalty deductions used in the health score computation (Equation 3.2).

3.2 Per-Anti-Pattern Detection Methodology

This section describes the detection algorithm for each anti-pattern targeted by this work. For each anti-pattern, the input data, detection logic, configurable thresholds, and evidence collection mechanism are presented. The definitions and motivations for each anti-pattern were introduced in Chapter 2; this section focuses on the *how* of detection.

3.2.1 Shared Database Detection

The Shared Database detector operates by comparing the datasource connection URLs configured across all detected microservices. The algorithm proceeds in two phases.

In the first phase, the configuration files of each microservice are parsed to extract the datasource URL. For Spring Boot applications, the detector reads `application.yml`, `application.yaml`, or `application.properties` configuration files located in the `/resources` directory of each service. From YAML files, the nested property `spring.datasource.url` is extract using a recursive map traversal, whereas from properties files, the same key is read directly. Spring propery placeholders

with default values (e.g., `${DB_HOST:localhost}`) are resolved to their defaults to enable meaningful comparison.

In the second phase, microservices are grouped by their resolved datasource URL. Any group containing more than one service indicates a shared database instance. Formally, let $\mathcal{S}$ denote the set of all microservices and dsUrl denote the resolved datasource URL of service s. The detector reports a Shared Database anti-pattern for each equivalence class:

$$\forall\, u \in URLs : \left|\{s \in \mathcal{S} : dsUrl(s) = u\}\right| > 1 \tag{3.3}$$

For each detected instance, the detector extracts a code snippet from the configuration file of each sharing service, highlighting the line containing the datasource declaration, for immediate evidence for remediation.

Algorithm 1 summarises the detection procedure.

Algorithm 1 Shared Database Detection

Require: Set of microservices $\mathcal{S}$, project root path P
Ensure: Set of detected Shared Database anti-patterns

```
urlMap ← ∅                                   ▷ Map: datasource URL → list of services
for each service s ∈ S do
    u ← ParseDatasourceUrl(P, s)
    if u ≠ null then
        urlMap[u] ← urlMap[u] ∪ {s}
    end if
end for
results ← ∅
for each (u, services) ∈ urlMap do
    if |services| > 1 then
        snippets ← ReadDatasourceSnippets(P, services)
        results ← results ∪ {SharedDB(u, services, snippets)}
    end if
end for
return results
```

3.2.2 Cyclic Dependency Detection

Cyclic dependency detection is performed on the directed dependency graph $G = (V, E)$ constructed in the inter-service analysis-phase (see Section 3.1.4). The detector applies Tarjan's algorithm for finding strongly connected components (SCCs) [**?**]. Any SCC containing more than one vertex represents a dependency cycle among the corresponding services.

The algorithm operates in $O(|V| + |E|)$ time. It performs a depth-first traversal of the graph, maintaing for each vertex v two values: index(v), the order in

which v was visited and lowlink(v), the smallest index reachable from v through the DFS subtree and back edges. A vertex v is the root of an SCC when lowlink(v) = index(v), at which point all vertices on the stack above v (inclusive) form the SCC.

Algorithm 2 presents the implementation of Tarjan's SCC algorithm as used in this work. The outer loop (lines 4–8) ensures all vertices are visited even in disconnected graphs.

Algorithm 2 Tarjan's SCC Algorithm for Cycle Detection

Require: Directed graph $G = (V, E)$
Ensure: List of strongly connected components
1: index $\leftarrow 0$; stack $\leftarrow \emptyset$; result $\leftarrow \emptyset$
2: idx$[v] \leftarrow$ undefined, low$[v] \leftarrow$ undefined for all $v \in V$
3: onStack$[v] \leftarrow$ `false` for all $v \in V$
4: **for** each $v \in V$ **do**
5: **if** idx$[v]$ is undefined **then**
6: StrongConnect(v)
7: **end if**
8: **end for**
9: **return** result

10: **procedure** StrongConnect(v)
11: idx$[v] \leftarrow$ index; low$[v] \leftarrow$ index; index $\leftarrow$ index $+ 1$
12: Push v onto stack; onStack$[v] \leftarrow$ `true`
13: **for** each $(v, w) \in E$ **do**
14: **if** idx$[w]$ is undefined **then**
15: StrongConnect(w)
16: low$[v] \leftarrow \min($low$[v],$ low$[w])$
17: **else if** onStack$[w]$ **then**
18: low$[v] \leftarrow \min($low$[v],$ idx$[w])$
19: **end if**
20: **end for**
21: **if** low$[v] =$ idx$[v]$ **then**
22: scc $\leftarrow \emptyset$
23: **repeat**
24: Pop w from stack; onStack$[w] \leftarrow$ `false`
25: scc $\leftarrow$ scc $\cup \{w\}$
26: **until** $w = v$
27: result $\leftarrow$ result $\cup$ {scc}
28: **end if**
29: **end procedure**

For each SCC with |scc| > 1, the detector constructs a human-readable cycle description (e.g., "Order Service $\rightarrow$ Payment Service $\rightarrow$ Order Service") and collects code snippets from the dependency evidence. The evidence is drawn from the `@FeignClient` annotations, `RestTemplate` invocations, or `WebClient` calls that established the edges in the cycle.

3.2.3 Nano Service Detection

The Nano Service detector identifies services whose size is too small to justify the operational overhead of running them as independent microservices [**?**]. Detection is based on two configurable metrics: the lines of code (LOC) and the number of REST endpoints exposed by the service.

A microservice s is flagged as a nano service when the following conjunction holds:

$$LOC(s) < \theta_{LOC} \ \wedge \ endpoints(s) \leq \theta_{ep} \tag{3.4}$$

where θ_{LOC} is the maximum lines of code threshold (default: 500, configurable via `app.thresholds.nano-service-max-loc`) and θep is the maximum endpoint count threshold (default: 2, configurable via `app.thresholds.nano-service--max-endpoints`).

The LOC count is computed the microservice detection phase by recursively walking through the service's directory tree and countine lines in all `.java` files. The endpoint count is populated during the inter-service analysis phase via Spoon-based annotation scanning.

As evidence, the detector attempts to locate the service's main application class (the class annotated with `@SpringBootApplication` or containing a `public static void main` method) and extracts a code snippet showing the first 15 lines, aiming to provide context about the service's entry point and overall structure.

3.2.4 God Service Detection

The God Service detector identifies microservices that handle too many responsibilities [**?**]. Rather than defining responsibility count heuristics directly, this detector uses the output of DesigniteJava's intra-service analysis: specifically, the frequency of *God Class* code smells within a service.

The rationale is that a microservice containing multiple God Class instances is likely spanning multiple domains withing a single deployable unit. The detection criterion is:

$$\left|\{c \in classes(s) : isGodClass(c)\}\right| \geq \theta_{god} \tag{3.5}$$

where θ_{god} is the minimum number of God Class smells required to flag the service (default: 3, configurable via `app.thresholds.god-service-min-domains`).

The detector queries the code smell repository for all smells associated with the microservice, filters for those with smell type "God Class", and counts the results. When the thresholds is exceeded, code snippets are extracted from the source files

of each God Class occurrences, centered on the class declaration or the line number reported by DesigniteJava.

This detection approach highlights the multi-level analysis contribution of this work: a code-level smell (God Class, detected by DesigniteJava) is used as a signal for an architectural-level anti-pattern (God Service).

3.2.5 Chatty Service Detection

The Chatty Service detector identifies pairs of services that communicate excessively through fine-grained remote calls [**?**, **?**]. Detection operates on the dependency graph edges, specifically on the `callCount` attribute annotated on each edge during the inter-service analysis phase.

During the dependency graph construction (see Section 3.1.4), each time an inter-service call is detected via `@FeignClient`, `RestTemplate`, or `WebClient`, the call count for the corresponding edge is incremented. The Chatty Service detector then queries for all dependency edges where the call count exceeds a configurable threshold:

$$callCount(s_1, s_2) \geq \theta_{chatty} \tag{3.6}$$

where θ_{chatty} is the minimum call count threshold (default: 5, configurable via `app.thresholds.chatty-service-min-calls`).

For each chatty edge, the detector reports both the source and target services as affected, and extracts code snippets from the evidence file and line number recorded during dependency graph construction. The evidence shows the `@FeignClient` interface or the `RestTemplate` invocation responsible for the excessive communication.

3.2.6 Hardcoded Endpoint Detection

The Hardcoded Endpoint detector scans Java code files for URL literals that indicate services are not using a service discovery mechanism [**?**]. The detection algorithm performs a line-by-line scan of all `.java` files within each service's `/src/main/java` directory.

The detector uses a configurable set of URL patterns to identify hardcoded endpoints. By default, the patterns include `http://`, `https://`, `localhost:`, and `127.0.0.1`. For each line that matches one of these patterns, the detector verifies the match occurs withing a string literal (i.e., enclosed in quotes) and extracts the full URL by use of a regular expression:

```
(https?://[\w.\-:]+[/\w.\-?=&#%]*)
```

Several categories of lines are excluded from the scan to reduce false positives:

- Comment lines (starting with `//`, `*`, or `/*`)
- Import statements (`import ...`)
- Package declarations (`package ...`)
- Test files (paths containing `test` or `Test`)

For each detected hardcoded URL, the detector records the file path, line number, the offending line of code and the extracted URL. Code snippets with surrounding context are read from the source file to provide evidence. The number of evidence snippets is capped at fie per service to avoid excessively large payloads.

3.2.7 Distributed Monolith Detection

The Distributed Monolith detector evaluates whether the system as a whole exhibits characteristics of tight coupling that negate the benefits of microservice decomposition [**?**, **?**]. The detection algorithm computes three metrics from the dependency graph and service configuration data, then applies a composite decision rule.

The three metrics are:

1. **Coupling coefficient** (C): The ratio of actual unique dependency edges to the maximum number of possible edges in a directed graph (Equation 3.7).
2. **Connected service ratio** (R): The fraction of services that participate in at least one dependency, as either source or target (Equation 3.8).
3. **Shared database count** (D): The number of distinct database URLs that are shared by more than one service, computed using the same grouping logic as the Shared Database detector.

$$C = \frac{|\text{unique edges}|}{|V| \times (|V| - 1)} \tag{3.7}$$

$$R = \frac{\left|\{v \in V : \exists\, e \in E \text{ involving } v\}\right|}{|V|} \tag{3.8}$$

The system is flagged as a distributed monolith when any of the following conditions holds:

$$\begin{aligned} C > 0.5 \quad &\vee \quad (R > 0.8 \wedge D > 0) \\ &\vee \quad (R > 0.8 \wedge C > 0.3) \end{aligned} \tag{3.9}$$

The composite rule captures three distinct manifestations of the anti-pattern; firstly, systems where more than half of all possible inter-service dependencies exist. Secondly, systems where nearly all services are connected and they additionally share databases. Lastly, systems where nearly all services are connected with a moderately high coupling coefficient.

The detection is performed when the system contains at least three microservices ($|V| \geq 3$), as smaller systems do not exhibit the structural characteristics of a distributed monolith.

When detected, the anti-pattern report includes all services as affected, along with the computed metric values and up to five code snippets from inter-service dependency evidence.

3.2.8 API Versioning Absence Detection

The API Versioning Absence detector checks whether microservices expose REST endpoints without a versioning strategy [**?**]. The detection operates on the endpoint data collected during the Spoon-based inter-service analysis phase.

During endpoint extraction, each detected endpoint is tested against a regular expression pattern that matches common API version indicators:

```
/v\d+[/.]
```

This pattern matches path segments such as `/v1/`, `/v2/`, or `/v1.` and sets a boolean flag `hasVersioning` on each endpoint entity.

The detector then queries all endpoints for each microservice. A service is flagged with the API Versioning Absence anti-pattern when *none* of its endpoints contain a version indicator:

$$versionedCount(s) = \left|\{e \in ep(s) : hasVer(e)\}\right| = 0 \tag{3.10}$$

As evidence, the detector locates the controller classes that declare unversioned endpoints and extracts the code snippets showing the `@RestController` or `@RequestMapping` annotations. This allows the developer to see where versioning prefixes (e.g., `/v1/`) could be added.

3.3 Summary

This chapter presented the detection methodology employed by this work. The analysis pipeline was described, starting from source code ingestion and microservice boundary detection, through intra-service code smell analysis (via Designite-

Java) and inter-service dependency extraction (via Spoon-based AST analysis), to dependency graph construction and anti-pattern detection.

The detailed detection algorithm for each of the eight targeted anti-patterns was presented. Shared Database detection compares datasource URLs across services (Section 3.2.1). Cyclic Dependency detection applies Tarjan's SCC algorithm on the dependency graph (Section 3.2.2). Nano Service detection uses LOC and endpoint count thresholds (Section 3.2.3). God Service detection leverages God Class code smell frequency via DesigniteJava (Section 3.2.4). Chatty Service detection analyzes dependency edge call counts (Section 3.2.5). Hardcoded Endpoint detection performs URL pattern matching in source files (Section 3.2.6). Distributed Monolith detection employs composite coupling metric analysis (Section 3.2.7). API Versioning Absence detection matches endpoint path versioning patterns (Section 3.2.8).

All detection thresholds are externalized as configuration parameters, allowing calibration to project-specific norms. Each detector produces evidence-based reports with source code snippets, enabling developers to quickly understand and remediate detected issues. The next chapter describes the implementation of these detection algorithms within the web application.

Chapter 4

Application Design & Implementation

This chapter presents the design and implementation of the MSA Detector, a web application for the automated detection of architectural anti-patterns in Java-based microservice systems. The chapter begins with a high-level system overview and technology stack summary, followed by the data model design. The backend implementation is described, covering the REST API, analysis pipeline orchestration, and security layer. The frontend implementation is then presented, followed by the deployment configuration.

4.1 System Overview

The MSA Detector follows a client–server architecture with three main components: a Spring Boot backend that exposes a RESTFul API [**?**], an Angular single-page application (SPA) that provides the user interface, and a PostgreSQL relational database for persistent storage. All components are containerized using Docker and orchestrated via Docker Compose, as can be seen in figure 4.1.

The system supports two modes of project ingestion: users can either upload a ZIP archive containing a microservices project, or provide a public GitHub/GitLab repository URL to be cloned. Once uploaded, the analysis pipeline runs asynchronously, detecting microservices boundaries, running static analysis, building the inter-service dependency graph, and detecting anti-patterns. Results are persistes and made available through the API, including a composite health score, detected anti-patterns with evidence and a dependency graph visualization.

4.1.1 Technology Stack

Table 4.1 summarizes the technologies used in each layer of the system.

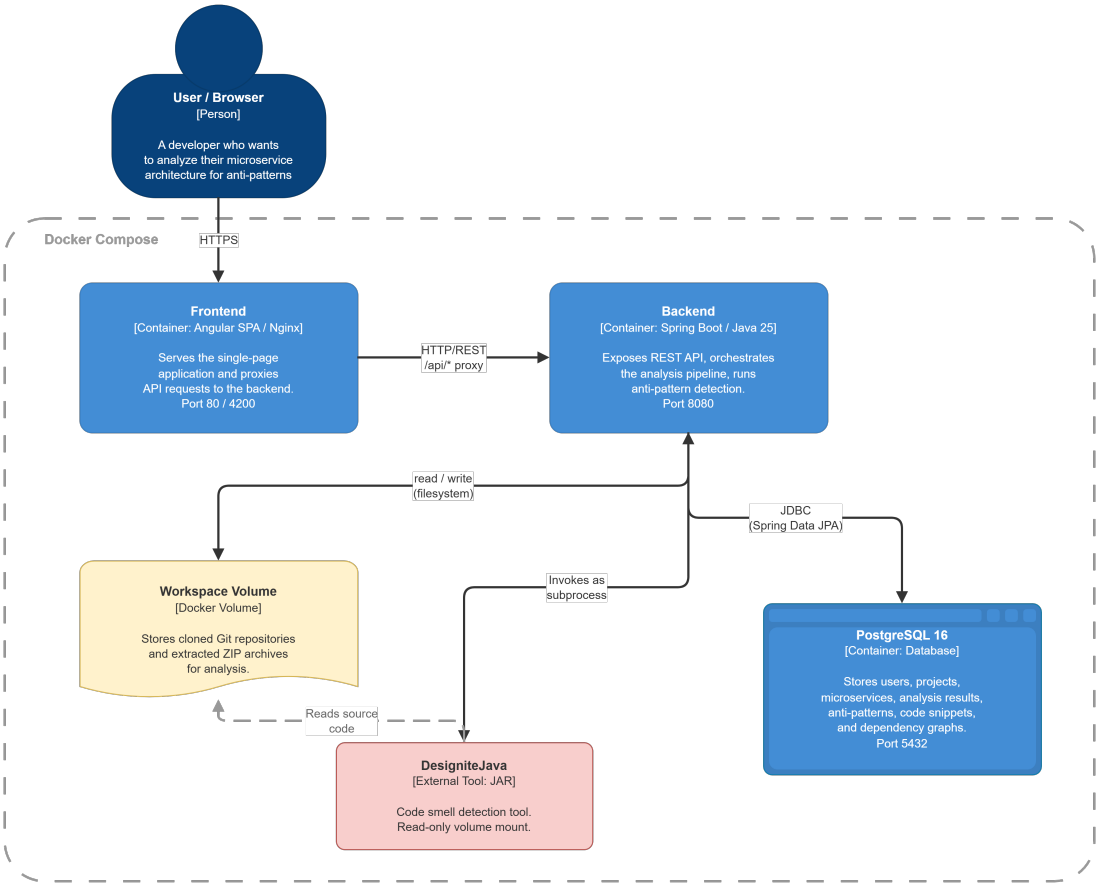


Figure 4.1: High-level architecture of the MSA Detector system. The Angular frontend communicates with the Spring Boot backend over HTTP/REST. The backend persists data in PostgreSQL and performs analysis on cloned/uploaded source code stored on the filesystem. All components run as Docker containers orchestrated via Docker Compose.

Layer	**Technology**	**Purpose**
Backend	Java 25, Spring Boot 4.0	Application framework
	Spring Data JPA, Hibernate	ORM and data access
	Spring Security, JWT (jjwt 0.12.6) [?]	Authentication and authorization
	Spoon 11.1.0 [?]	AST-based Java source code analysis
	DesigniteJava [?]	Code smell detection
	JGit 7.1.0	Git repository cloning
	Flyway	Database schema migrations
	MapStruct 1.6.3, Lombok 1.18.38	DTO mapping and boilerplate reduction
Frontend	Angular 21	Single-page application framework
	TypeScript	Programming language
Database	PostgreSQL 16	Relational data storage
Infrastructure	Docker, Docker Compose	Containerization and orchestration
	Eclipse Temurin 25 (Alpine)	JDK/JRE base images
API Documentation	SpringDoc OpenAPI 2.7.0	Swagger UI and OpenAPI spec
Testing	JUnit 5, Testcontainers 1.20.4	Unit and integration testing

Table 4.1: Technology stack summary for the MSA Detector system.

4.2 Data Model

The persistent data model consists of nine entities managed via JPA and stored in PostgreSQL. All entities extend a common `BaseEntity` superclass that provides an auto-generated primary key (`id`), a creation timestamp (`created_at`) and a last-modified timestamp (`updated_at`) via JPA auditing. The database schema is defined and versioned using Flyway migrations.

4.2.1 Entity Descriptions

Table 4.2 provides an overview of each entity and its role in the system. The relationships between entities are described below.

4.2.2 Entity Relationships

The entity relationships form a hierarchical structure with the `User` entity as root. A `User` owns multiple `Project` entities (one-to-many), with cascade delete removing all projects when a user is deleted. Each `Project` contains multiple detected `Microservice` entities and multiple `AnalysisJob` entities, both with cascade delete and orphan removal. A `Microservice` entity exposes zero or more `Endpoint` entities, maintains outgoing `ServiceDependency` edges to other microservices and contains `CodeSmell` records. Each `ServiceDependency` references both a source and a target `Microservice`, representing a directed edge in the dependency graph. Every completed `AnalysisJob` produces exactly one `AnalysisResults` (one-to-one), which contains zero or more `DetectedAntiPattern` instances. A `DetectedAntiPattern` optionally references a primary `Microservice` for service-scoped anti-patterns such as Nano Service or God Service.

All foreign key relationships use `ON DELETE CASCADE` at the database level, thus ensuring that deleting a project removes all associated microservices, endpoints, dependencies, code smells, analysis jobs, results and detected anti-patterns.

4.3 Backend Implementation

The backend is implemented as a Spring Boot application structured into the standard layers: controllers (REST API), services (business logic), repositories (data access), entities (domain model), DTOs (data transfer objects) and cross-cutting concers (security, configuration, exception handling).

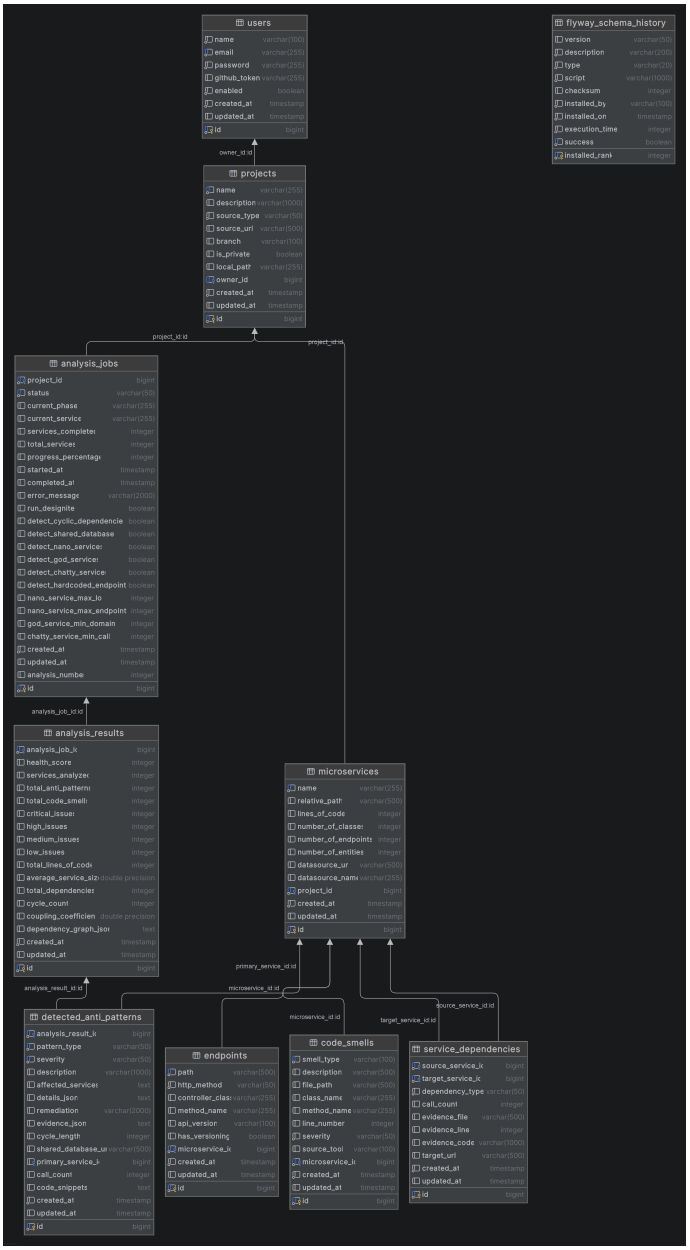


Figure 4.2: High-level architecture of the MSA Detector system. The Angular frontend communicates with the Spring Boot backend over HTTP/REST. The backend persists data in PostgreSQL and performs analysis on cloned/uploaded source code stored on the filesystem. All components run as Docker containers orchestrated via Docker Compose.

Entity	Description
`User`	Registered user with name, email, hashed password, and optional GitHub token. Owns zero or more projects.
`Project`	A microservice project with a name, source type (`GITHUB`, `GITLAB`, or `UPLOAD`), optional source URL and branch, and local filesystem path. Owned by a user. Contains zero or more microservices and analysis jobs.
`Microservice`	A detected microservice within a project. Stores the service name, relative path, lines of code, number of classes, number of endpoints, and datasource configuration.
`Endpoint`	A REST endpoint detected within a microservice. Stores the URL path, HTTP method, controller class, method name, and API versioning information.
`ServiceDependency`	A directed dependency edge between two microservices. Stores the dependency type (e.g., `FEIGN_CLIENT`, `REST_TEMPLATE`, `WEB_CLIENT`), call count, and evidence (file path, line number, code snippet, target URL).
`CodeSmell`	A code smell detected within a microservice by Designite-Java. Stores the smell type, severity, file path, class name, method name, and line number.
`AnalysisJob`	Represents a single analysis execution for a project. Tracks the job status through its lifecycle (`PENDING` → `DETECTING_SERVICES` → `ANALYZING_SERVICES` → `BUILDING_GRAPH` → `DETECTING_PATTERNS` → `COMPLETED` or `FAILED`), progress percentage, configurable detection flags and thresholds, and the analysis number for re-analysis tracking.
`AnalysisResult`	The output of a completed analysis job. Stores aggregate metrics (health score, services analyzed, total anti-patterns, code smell count, issue counts by severity, total LOC, average service size, dependency count, cycle count, coupling coefficient) and the dependency graph as a JSON string. Has a one-to-one relationship with `AnalysisJob`.
`DetectedAntiPattern`	An individual anti-pattern instance found during analysis. Stores the anti-pattern type, severity, human-readable description, affected services (JSON), remediation advice, evidence (JSON), and type-specific fields such as cycle length, shared database URL, call count, and code snippets (JSON).

Table 4.2: Overview of the data model entities.

4.3.1 REST API Design

The backend exposes a RESTful API [**?**] under the `/api` prefix. The API is organized into three controller groups, summarized in Table 4.3.

Endpoint	Method	Description
Authentication (`/api/auth`)		
`/register`	POST	Register a new user
`/login`	POST	Authenticate and obtain JWT token
`/me`	GET	Get current authenticated user info
Projects (`/api/projects`)		
`/upload`	POST	Upload ZIP and trigger analysis
`/clone`	POST	Clone Git repo and trigger analysis
`/`	GET	List all projects for current user
`/{id}`	GET	Get project details
`/{id}`	DELETE	Delete project and all associated data
`/{id}/history`	GET	Get full analysis history with diffs
`/{id}/reanalyze`	POST	Re-analyze project (optionally switch source)
`/{id}/reupload`	POST	Upload new ZIP for existing project
Jobs (`/api/jobs`)		
`/{id}`	GET	Get job status and progress
`/{id}/results`	GET	Get analysis results with health score breakdown
`/{id}/diff`	GET	Get diff against previous analysis
`/recent`	GET	Get 20 most recent jobs for current user
`/{id}/cancel`	POST	Cancel a running job

Table 4.3: REST API endpoints exposed by the backend.

All endpoints except `/api/**`, Swagger UI, and the actuator health endpoint require JWT authentication. Request and response payloads use Java `record` DTOs serialized to JSON via Jackson, with null fields excluded from responses.

4.3.2 Authentication and Security

The system uses stateless JWT based authentication [**?**]. The security layer is built on Spring Security and consists of five components. The `SecurityConfig` class configures stateless session management, disables CSRF protection (the token-based API does not rely on cookies), and defines the authorization rules: public endpoints such as `/api/auth/**`, Swagger UI, and the actuator health check are exempted from authentication, while all other requests require a valid token. The `JwtTokenProvider` component generates and validates tokens using the HMAC-SHA algorithm [**?**] - each token encodes the user ID and email as claims and carries a configurable expiration (default: 24 hours). Listing 4.3 illustrates the token generation logic.

On every incoming request, the `JwtAuthenticationFilter`, a `OncePerRequestFilter`, extracts the JWT from the `Authorization: Bearer` header,

Figure 4.3: JWT token generation in the `JwtTokenProvider` class. The token encodes the user ID as the subject, the email as a custom claim, and is signed with an HMAC-SHA key.

delegates to the provider for validation, loads the corresponding user details via the `CustomUserDetailsService` and populates the Spring `SecurityContext`. If no valid token is present, the `JwtAuthenticationEntryPoint` returns a `401 Unauthorized` response. Passwords are hashed using BCrypt, and the authenticated user's identity is injected into controller methods via the `@AuthenticationPrincipal` annotation, which provides a `UserPrincipal` object containing the user's ID, name, and email.

4.3.3 Project Ingestion

The web application supports two modes of project ingestion, both handled by the `ProjectService`.

ZIP Upload: Users upload a ZIP archive via a multipart form data request. The file is validated (must be a ZIP, non-empty, under 500 MB), extracted to a project specific directory under the configured workspace path and normalized to handle archives that contain a single root directory. The file structure is protected against path traversal attacks (zip slip [1]) by verifying that all extracted paths remain within the target directory.

Git Clone: Users provide a public GitHub or GitLab HTTPS URL and an optional branch name. The URL is validated against a whitelist pattern. The `GitCloneService` uses JGit to perform a shallow clone (`depth=1`) of the repository, which downloads only the latest commit to minimize bandwith and disk usage. A configurable timeout (default: 300 seconds) prevents indefinite blocking.

Both methods create a `Project` entity and an `AnalysisJob` entity, then trigger the asynchronous analysis. To ensure data consistency, the asynchronous worker is not started immediately, instead, it is registered through Spring's `TransactionSynchronization.afterCommit()` callback, which guarantees that the job entity is fully committed to the database before the analysis thread begins reading it. For reanalysis, the application supports re-cloning a Git repository (to pick up new commits), re-uploading a new ZIP file, or switching the project's source type entirely (e.g. from upload to Git repository).

[1]

4.3.4 Analysis Pipeline Orchestration

The analysis pipeline is orchestrated by the `AnalysisWorker` service, which executes asynchronously via Spring's `@Async` mechanism backed by a configurable thread pool (core size 2, maximum 5). The pipeline proceeds through five phases, each updating the job's status and progress for real-time tracking by the frontend. Listing 4.4 shows the top-level orchestration method.

Figure 4.4: Simplified `AnalysisWorker.processJob()` method showing the five-phase analysis pipeline. Each phase updates job progress for frontend polling.

In the first phase, **Microservice Detection** (`DETECTING_MICROSERVICES`), the `MicroserviceDetector` scans the project directory for build files (`pom.xml`), `build.gradle`, `build.gradle.kts`) to identify individual microservice modules. It filters out non-service directories (e.g. examples, tests, documentation, benchmarks, build support modules, and templates) using an exclusion set of over 40 keywords matched against directory name components. For multi-module Maven or Gradle projects, it detects submodules by checking for `<modules>` declarations in `pom.xml` or `include` statements in `settings.gradle`. For single-module projects where no submodules are detected, the root directory is treated as the only service. For each detected microservice, a `Microservice` entity is created with the service name, relative path, and lines of code count computed by recursively walking all `.java` files.

The second phase, **Intra-Service Analysis** (`ANALYZING_SERVICES`), invokes DesigniteJava as a subprocess for each microservice (when enabled). The service passes the microservice's source directory to DesigniteJava, which produces CSV output files (`DesignSmells.csv`, `ImplementationSmells.csv`, `ArchitectureSmells.csv`). These files are parsed line by line and the detected smells are stored as `CodeSmell` entities with severity levels mapped from the smell type. The number of classes is extracted from the `TypeMetrics.csv` output. A configurable timeout (default: 1800 seconds) prevents runaway hanging subprocesses. Progress is reported per-service, allowing the frontend to display which service is currrently being analyzed and what fraction is complete.

During the third phase, **Dependency Graph Construction** (`BUILDING_GRAPH`), the `DependencyGraphBuilder` scans each microservice's source code using the Spoon static analysis library [**?**] to extract REST endpoints, parse datasource configuration from `application.yml` and `application.properties` and detect inter-service calls via `@FeignClient`, `RestTemplate`, and `WebClient`. The target resolution strategy, described in detail in Chapter 3, Section 3.1.4, uses the exact

name matching, port-based matching and fuzzy substring matching to resolve call targets to actual microservices. The resulting `Endpoint` and `ServiceDependency` entities are persisted to the databse.

The fourth phase, **Anti-Pattern Detection** (`DETECTING_PATTERNS`), delegates to the `AntiPatternDetectorService`, which orchestrates all registered detectors (see Section 4.3.5). Each detector is invoked in sequence, receiving the project and the list of microservices, then returning a list of `DetectedAntiPattern` instances. Failed detectors are logged and skipped, ensuring that a failure in one does not abort the entire analysis.

Finally, in the **Result Assembly** phase, summary statistics are computed (e.g. total lines of code, average service size, coupling coefficient (see Chapter 3, Equation 3.1), issue counts by severity) and the health score is calculated following the methodology described in Chapter 3, Section 3.1.5. The dependency graph is serialized to a JSON structure containg nodes (services with metadata) and edges (dependencies with type and weight). The `AnalysisResult` entity is persisted, and the job status is set to `COMPLETED`.

If any phase fails, the exception is caught, the job status is set `FAILED` with the error messsage and the failure is logged. Users may also cancel a running job via the API, which sets the status to `CANCELLED`. The `JobProgressUpdated` service provides atomic progress updates by executing each update in an independent transaction (`Propagation.REQUIRES_NEW`), thus ensuring that progress changes are immediately visible to polling clients regardless of the outer analysis transaction.

4.3.5 Anti-Pattern Detector Architecture

The anti-pattern detection subsystem follows the Strategy design patterns [**?**]. All detectors implement the `AntiPatternDetector` interface, which defines a single method as shown in Listing 4.5.

Figure 4.5: The `AntiPatternDetector` interface. Spring auto-collects all implementations via `List<AntiPatternDetector>` dependency injection.

A `BaseDetector` abstract class provides shared utility methods for JSON serialization, string truncation and source code snippet extraction. The snippet extraction method reads a configurable number of context lines above and below a target line number from the source file, producing a map with the file path, highlighted line, and surrounding code. This evidence is stored as JSON in the `DetectedAntiPattern` entity and rendered by the frontend to give developers immediate insight into the detected issue.

Table 4.4 lists all eight implemented detectors. The detection algorithms for each anti-pattern were described in Chapter 3. New detectors can be added by implementing the interface and annotating the class with `@Component`; Spring's dependency injection automatically discovers and registers them with the orchestrator, requiring no changes to the `AntiPatternDetectorService`.

Detector Class	Anti-Pattern	Severity
`SharedDatabaseDetector`	Shared Database	HIGH
`CyclicDependencyDetector`	Cyclic Dependency	CRITICAL
`NanoServiceDetector`	Nano Service	MEDIUM
`GodServiceDetector`	God Service	HIGH
`ChattyServiceDetector`	Chatty Service	HIGH
`HardcodedEndpointDetector`	Hardcoded Endpoints	MEDIUM
`DistributedMonolithDetector`	Distributed Monolith	CRITICAL
`ApiVersioningDetector`	API Versioning Absence	MEDIUM

Table 4.4: Implemented anti-pattern detectors with their target patterns and default severity levels. The detection algorithms for each anti-pattern were described in Chapter 3.

4.3.6 Analysis Diff (Re-Analysis Comparison)

When a project is re-analyzed, the backend computes a diff between the current and previous analysis results, inspired by SonarQube's "Clean as You Code" concept [**?**]. The `AnalysisDiffService` compares two `AnalysisResult` instances and produces an `AnalysisDiffResponse` DTO.

The diff includes the health score delta, previous versus current score and letter grade, as well as per-metric deltas for all aggregate metrics: total anti-patterns, code smells, issues by severity, total lines of code, services analyzed, cycle count, dependency count and coupling coefficient. Anti-pattern changes are categorized in three groups: *resolved* anti-patterns that were present in the previous analysis but absent in the current one, *new* anti-patterns detected for the first time, and *unchanged* anti-patterns that were still found in the later runs. The diff also computes per-category score deltas for each of the four health score categories (Anti-Patterns, Code Quality, Architecture, Service Sizing). Finally, a human-readable summary string is generated (e.g., "Health improved from 52 (D) $\rightarrow$ 78 (C). 2 anti-patterns resolved, 1 new."), providing a short assessment of the changes.

The diff is automatically embedded in the analysis results response and is also available through a dedicated `GET /api/jobs/{id}/diff` endpoint. The project history endpoint returns all completed analyses ordered from the newest first, each paired with its diff against the predecessor, thus providing a full timeline view of architectural evolution.

4.3.7 Exception Handling

A global exception handler (`GlobalExceptionHandler`) annotated with Spring's `@RestControllerAdvice` provides a centralized error handling across all controllers. The handler translated domain exceptions into structured JSON error responses with a consistent format containing an error code, a human-readable message, and a timestamp. `ResourceNotFoundException` maps to `404 Not Found`; `InvalidFileException` and `IllegalArgumentException` map to `400 Bad Request`; `IllegalStateException` maps to `409 Conflict` for operations on resources in an invalid state (e.g., cancelling an already-completed job); `BadCredentialsException` maps to `401 Unauthorized`; and `MethodArgumentNotValidException` maps to `400 Bad Request` with per-field validation messages. A catch-all handler logs unexpected exceptions at error level and returns a generic `500 Internal Server Error` to avoid leaking stack traces.

4.4 Frontend Implementation

The frontend is implemented as an Angular single-page application (SPA) written in TypeScript. It communicates with the backend exclusively through the REST API described in Section 4.3.1. This section describes the application structure, key pages, reusable components and the authentication integration.

4.4.1 Application Structure and Routing

The frontend is built with Angular 19 and uses the framework's standalone component architecture - the application contains no `NgModule` declarations. Every component, page, interceptor and guard is defined as a standalone unit that declares its own imports, simplifying the dependency graph and enabling fine-grained tree-shaking at build time.

The application is bootstrapped in `main.ts` using Angular's `bootstrapApplication` function, which receives the root `AppComponent` and a providers array. The providers array registers the router (via `provideRouter`) and the HTTP client (via `provideHttpClient` with `withInterceptors`), injecting both the authentication and error interceptors into the HTTP pipeline at bootstrap time.

The source tree is organized into five directories under `src/app/`. The `pages/` directory contains five page-level components, each corresponding to a route. The `components/` directory contains seven reusable presentational components that are composed into pages via Angular's input binding mechanism. The `services/` directory contains two singleton services: `ApiService`, which encapsulates all

HTTP calls to the backend REST API, and `AuthService`, which manages JWT-based authentication state. The `interceptors/` directory contains two functional HTTP interceptors, and the `guards/` directory contains a single functional route guard. Finally, the `models/` directory provides a barrel file exporting all TypeScript interfaces and type aliases used throughout the application.

The root `AppComponent` renders a persistent navigation header and a `<router-outlet>` for the active page. The header is context-sensitive: when the user is authenticated, it displays navigation linkes to the Upload and History pages, the current user's name (rendered via the `currentUser$` observable with Angular's `async` pipe) and a Sign Out button. When the user is not authenticated, it shows Sign In and Sign Up links instead.

Table 4.5 lists the route configuration. The empty path redirects to `/upload`, so authenticated users land on the upload page by default. Three of the five routes are protected by the `authGuard`, which redirects unauthenticated users to the login page.

Path	Component	Auth Required
`/login`	`LoginPageComponent`	No
`/register`	`RegisterPageComponent`	No
`/upload`	`UploadPageComponent`	Yes
`/results/:jobId`	`ResultsPageComponent`	Yes
`/history`	`HistoryPageComponent`	Yes

Table 4.5: Frontend routing configuration. Routes marked "Auth Required" are protected by the `authGuard`.

4.4.2 Authentication Integration

The frontend authentication layer consists of four cooperating pieces: a service that manages token storage and user state, two HTTP interceptors that transparently handle token attachment and session expiration, and a route guard that protects authenticated pages.

The `AuthService` is a singleton service provided at the root level. It stores the JWT token and the serialized user object in the browser's `localStorage` under the keys `auth_token` and `auth_user`, respectively. On construction, the service attempts to load a previously stored used from `localStorage` and initializes `BehaviorSubject<UserResponse | null>`, which is exposed as the `currentUser$` observable. This observable is consumed by the root `AppComponent` to reactively render the navigation bar. The service provides `login()` and `register()` methods that POST credentials to the backend's `/api/auth/login` and `/api/auth/register` endpoints, respectively; both methods pipe the response through a `tap` operator that persists the returned token and user object. The `logout()` method clears

both `localStorage` entries and pushes `null` into the subject, which immediately updates all subscribed components. The `isAuthenticated()` method performs a simple null check on the stored token and is used by both the route guard and the navigation bar.

The `authInterceptor` is implemented as a functional `HttpInterceptorFn`. For every outgoing HTTP request, it injects the `AuthService` via Angular's `inject()` function and retrieves the current token. Requests to the `/auth/login` and `/auth/register` endpoints are passed through unmodified, since those endpoints are public. For all other requests, if a token is present, the interceptor clones the request with an `Authorization: Bearer <token>` header attached.

The `errorInterceptor` is also a functional `HttpInterceptorFn`. It wraps the downstream handler in an RxJS `catchError` operator. When a response with 401 status is received (and the request URI does not contain `/auth/`), the interceptor calls `authService.logout()` to clear the stale sessions and redirects the user to `/login`. All other errors are re-thrown so that individual components can handle them as needed.

The `authGuard` is implemented as a functional `CanActivateFn`. It checks `authService.isAuthenticated()`: if the check passes, the guard returns `true` and the navigation proceeds. Otherwise, it redirects to `/login` and returns `false`. The guard is applied to the `/upload`, `/results/:jobId`, and `/history` routes.

4.4.3 Pages

The application contains five page components, each responsible for a distinct user workflow.

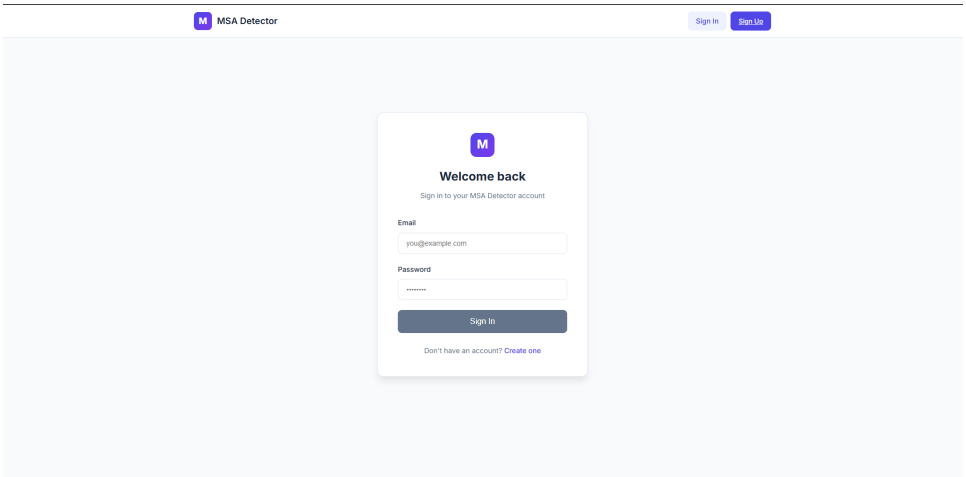
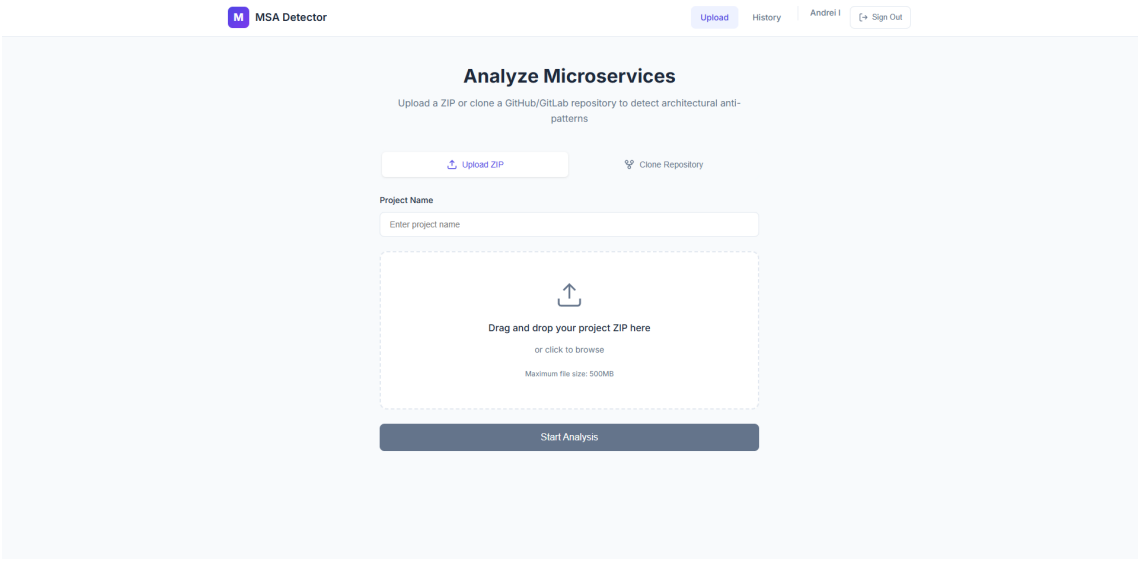


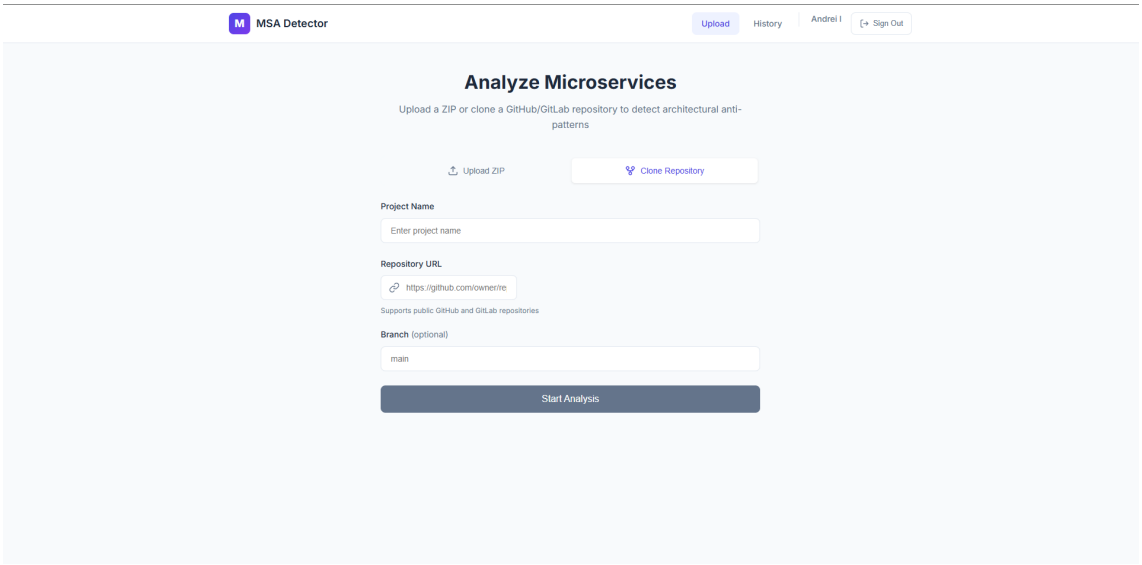
Figure 4.6: Login page of the MSA Detector frontend.

Login and Register Pages: The `LoginPageComponent` presents a form with email and password fields, bound via Angular's `FormsModule` two-way binding

(`ngModel`). A computed `canSubmit` getter disables the submit button until both fields are populated and no request is in flight. On submission, the component calls `AuthService.login()` and navigates to `/upload` on success. Conversely, on failure, it extracts the error message from the backend response and display it inline. The `RegisterPageComponent` follows the same pattern but adds a name field and enforces minimum password length of six characters. Both pages include a navigation link to switch between login and registration.



(a) ZIP upload 1



(b) Git clone 2

Figure 4.7: Upload page with ZIP upload (left tab) and Git clone (right tab) modes.

Upload Page: The `UploadPageComponent` supports two project ingestion modes, selectable via a tab interface. The active mode is tracked by the `activeMode` property, which is either `'upload'` or `'clone'`.

In ZIP upload mode, the page embeds the reusable `FileUploadComponent`

(described in Section 4.4.4) for drag-and-drop file selection. When a file is selected, the project name field is automatically populated from the filename by stripping the `.zip` extension. On submission, the component calls `ApiService.uploadProject()`, which sends a multipart form data request. Upload progress is tracked in real time by observing `HttpEvent.UploadProgress` events from Angular's `HttpClient`, updating a progress percentage that is displayed to the user.

In Git clone mode, the page presents a repository URL input and an optional branch field. The URL is validated client-side using a regular expression that accepts HTTP URLs for `github.com` and `gitlab.com`. When a URL is entered, the project name is extracted from the last path segment. On submission, the component calls `ApiService.cloneProject()` with the URL, project name, and optional branch.

Both submission paths follow the same post-submission flow. Once the backend responds with a job ID, the page transitions to an analysis-in-progress state and embeds `ProgressTrackerComponent`. A polling loop, implemented with `setInterval` at a 2 second interval, repeatedly calls `ApiService.getJobStatus()` to fetch the current job status and progress. When the job status changes to `COMPLETED`, the polling is stopped and the router navigates to the results page. If the status is `FAILED` or `CANCELLED`, the polling stops and the error message is displayed. The polling interval is cleared in the component's `ngOnDestroy` lifecycle hook to prevent memory leaks on navigating away.

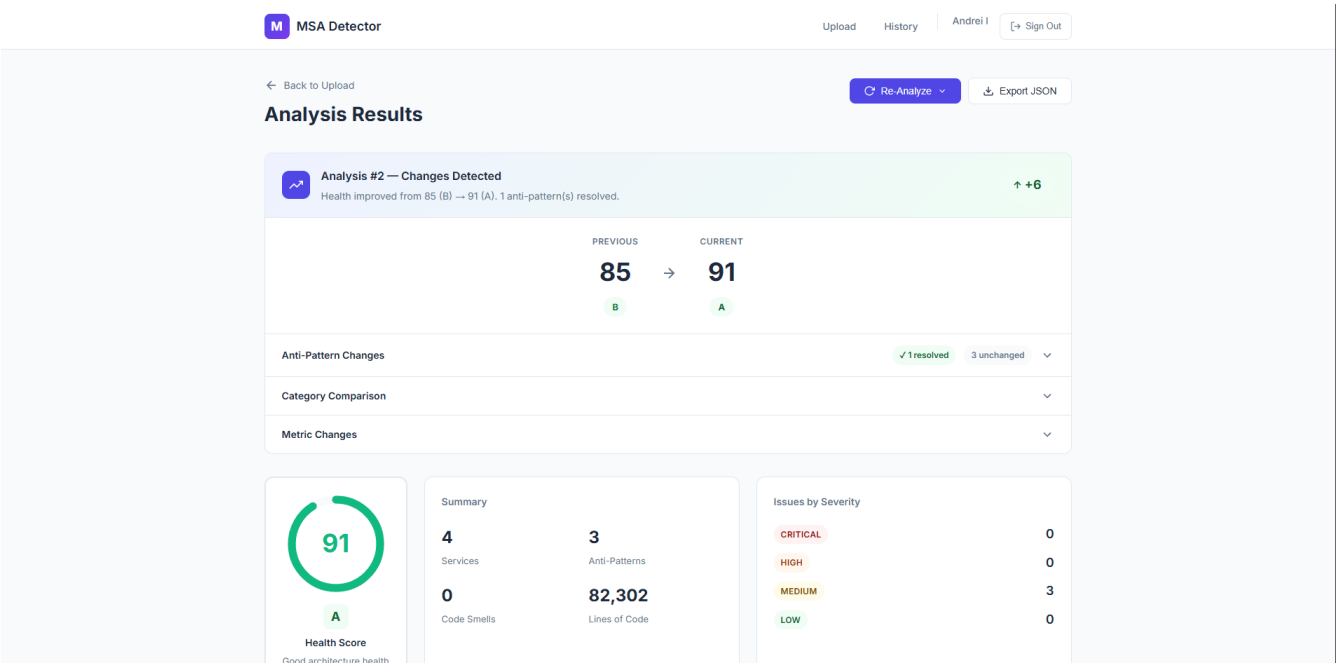


Figure 4.8: Results page displaying the health score gauge, summary metrics, issues by severity, health score breakdown, detected anti-patterns, and dependency graph.

Results Page: The `ResultsPageComponent` is the primary output of the application. It reads the `jobId` parameter from the route, calls `ApiService.getJobResults()`

to load the `AnalysisResult`, and then chains two additional requests to retrieve the job metadata and the parent project.

The page layout is organized into several sections. At the top, a header displays the project name, a `Re-Analyze` dropdown button, and an `Export JSON` button. The dropdown allows users to select one of two options: upload a new ZIP archive (which opens a modal dialog embedding the `FileUploadComponent`) or clone from a different GitHub or GitLab URL (which opens a modal dialog with URL and branch fields, pre-filled from the project's current source configuration). When any re-analysis option is selected, the page transitions to a re-analysis-in-progress state, diplaying the `ProgressTrackerComponent` and beginning polling the new job. On completion, the page reloades the results in place. The export button serializes the entire `AnalysisResult` object to a JSON blob and triggers a browser download with the filename `analysis-results0{jobId}.json`.

If the result contains a diff (i.e. the analysis is a re-analysis with a predecessor job), the `DiffBannerComponent` is rendered at the top of the page, showing how metrics have changed since the previous analysis. For first-time analyses, an informational note is rendered instead, explaining that re-analysis can be done later to track changes over time.

Below the diff banner, a grid of summary cards is displayed. The first card containsthe `HealthScoreComponent`, which renders the composite health score as a circular gauge with a letter grade badge. The second card shows four key metrics: services analyzed, total anti-patterns, total code smells and lines of code. The third card breaks down issue counts by severity (critical, high, medium, low), each being displayed with a color-coded badge.

Below the summary grid, the `HealthScoreBreakdownComponent` renders the per-category score breakdown as collapsible cards. Finally, a two-column details grid displays the `AntiPatternListComponent`, which is a list of detected anti-patterns that can be expanded to reveal descriptions, affected services, code evidence and remediation advice, alongside the `DependencyGraphComponent`, which is an interactive visualization of the service dependency graph.

History Page: The `HistoryPageComponent` provides an overview of all the user's projects and recent analysis jobs, organized into two tabs: `Projects` and `Jobs`.

The Projects tab loads the user's projects via `ApiService.getProjects()` and renders them as a list. Each project entry shows its name, source type (i.e. Git or ZIP), creation date and number of analyses. Clicking a project expands it to reveal the full analysis history, loaded via `ApiService.getProjectHistory()`. Each historical analysis entry shows its health score, anti-pattern count and, when available, the diff against the previous analysis (e.g. "+1 new anti-pattern, 2 resolved").

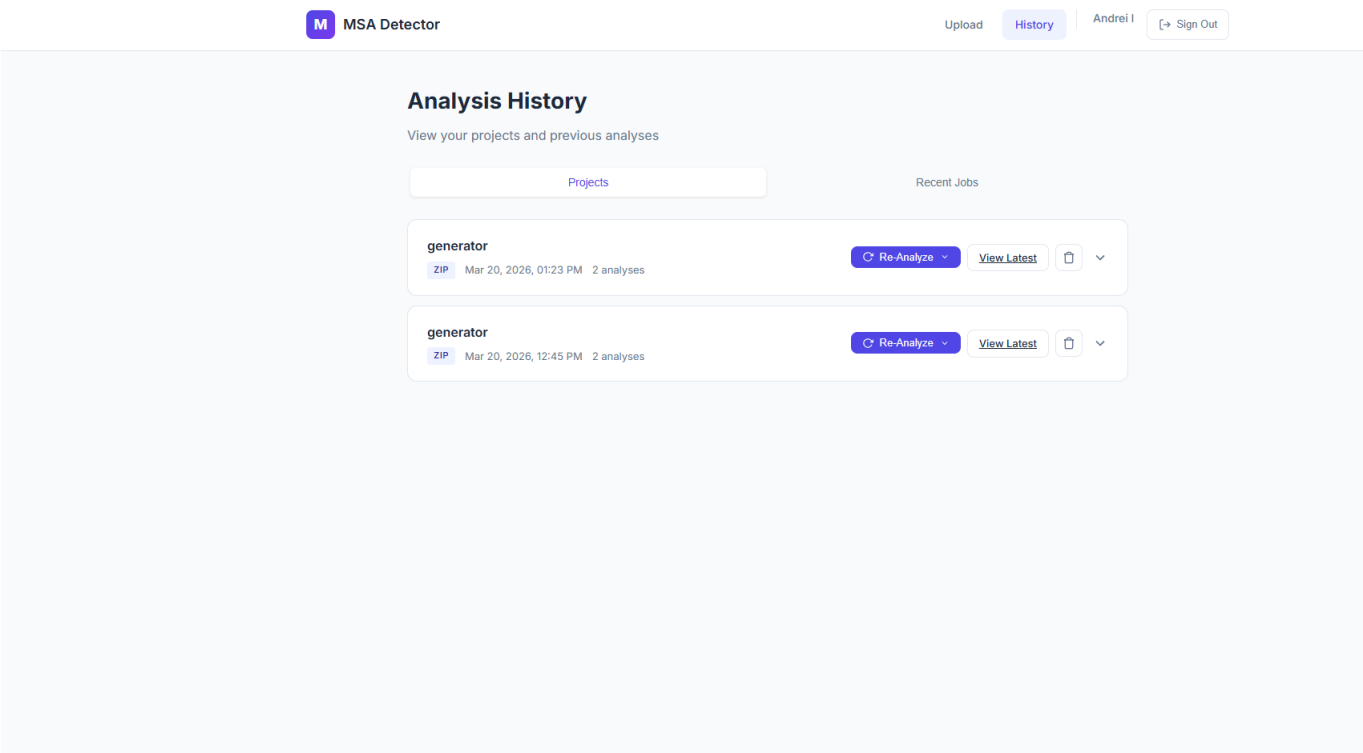


Figure 4.9: History page showing the projects list with an expanded project's analysis timeline and diff information.

Clicking an analysis entry navigates to the results page for that job. The health score delta is displayed with color coding: green for improvement, red for regression. Each project has a dropdown menu offering two re-analysis options (new ZIP upload, Git clone), following the same pattern as the results page. Projects can also be deleted, with a confirmation dialog displayed via the browser's native `confirm()` function. Deletion calls `ApiService.deleteProject()` and removes the project from the local list on success.

The Jobs tab displays the 20 most recent analysis jobs, loaded via `ApiService.getRecentJobs()`. Each entry shows the job ID, status (formatted by replacing underscores with spaces), analysis number, and timestamps.

4.4.4 Reusable Components

The application defines seven standalone reusable components that are composed into pages via Angular's input and output bindings. This section describes each component's purpose and implementation.

Health Score Display (`HealthScoreComponent`): This component accepts a numeric `score` (0 - 100) and an optional letter `grade` as inputs. It renders a circular gauge using an SVG circle with a `stroke-dashoffset` animation. The SVG consists of two overlapping circles with a radius of 45 units inside a 100×100 view-

Box: a background circle and a progress circle whose visible arc length is computed as `circumference − (score / 100) × circumference`, where the circumference is $2\pi r \approx 283$ units. The score value is displayed numerically in the center of the circle. The component applies one of three CSS classes based on the score: "good" (score ≥ 80), "warning" ($50 \leq$ score < 80), or "danger" (score < 50), which control the stroke and text color. Below the gauge, the letter grade is shown as a color-coded badge, and a textual description summarizes the score range.

Health Score Breakdown (`HealthScoreBreakdownComponent`): This component accepts a `HealthScoreBreakdown` object containing the overall grade and an array of `ScoreCategory` entries (each with a name, description, score, maximum score, and list of deductions). It renders each category as a collapsible card. The collapsed state shows the category name, a fractional score label (e.g., "35 / 40"), and a horizontal progress bar whose fill width is computed as a percentage of the maximum score. The bar is color-coded using the same three-tier scheme as the health score gauge. Clicking a card toggles its expanded state, which reveals the category's textual description and, if deductions are present, a list of deduction rows showing the reason and points lost. Categories with no deductions display a "Perfect score — no issues found" message.

Diff Banner (`DiffBannerComponent`): This component accepts an `AnalysisDiffResponse` object and renders a comparison between the current and previous analyses. The top section displays the health score delta with a directional arrow icon (up for improvement, down for regression, right for no change) and the previous and current letter grades, each styled with a grade-specific CSS class. Below this, a grid of eleven metric cards displays the delta for each tracked metric: anti-patterns, code smells, issues by each severity level, lines of code, services analyzed, cycle count, dependencies, and coupling coefficient. Each card shows the previous value, current value, and the signed delta. The component supports an `inverse` mode for metrics where a decrease is an improvement (e.g., fewer anti-patterns), ensuring that color coding is applied correctly. Collapsible sections list the resolved, new, and unchanged anti-patterns, as well as per-category score deltas. A human-readable summary string generated by the backend is displayed at the bottom.

Dependency Graph Visualization (`DependencyGraphComponent`): This component accepts a `DependencyGraph` object containing arrays of nodes (microservices) and directed edges (dependencies) and renders an interactive graph using Cytoscape.js (version 3.10) [2], a graph theory library for visualization and analysis. Each node represents a microservice, with its visual diameter scaled by lines of code (clamped between 30 and 60 pixels). Edges are drawn as directed bezier curves with triangle arrowheads. All node and edge IDs are coerced to strings be-

[2]

fore being passed to Cytoscape to prevent silent edge drops caused by type mismatches between numeric JSON IDs and Cytoscape's string-based ID system. The graph uses the CoSE (Compound Spring Embedder) force-directed layout algorithm with configurable ideal edge length (150 pixels) and node repulsion (8000 units). A `Reset View` button calls Cytoscape's `fit()` method to re-center and re-zoom the viewport. The graph canvas supports interactive panning and zooming via mouse events. The Cytoscape instance is properly destroyed and re-created on input changes (via `ngOnChanges`) and on component destruction (via `ngOnDestroy`) to avoid memory leaks and stale state.

File Upload (`FileUploadComponent`): This component provides a drag-and-drop file selection zone. It emits a `fileSelected` event (via Angular's `@Output` decorator) whenever a file is selected or removed. The drop zone listens for `dragover`, `dragleave`, and `drop` DOM events, applying a visual highlight during drag hover. A hidden `<input type="file" accept=".zip">` element is triggered on click as a fallback for users who prefer a file browser dialog. Only files with the `.zip` extension are accepted. Once a file is selected, the zone displays the filename, the file size (formatted in human-readable units: B, KB, or MB), and a remove button that clears the selection and emits `null`.

Code Snippet Viewer (`CodeSnippetViewerComponent`): This component accepts an array of `CodeSnippet` objects and renders source code evidence inline. Each snippet shows the file path (split into a directory prefix and a filename for visual clarity), followed by the code block rendered in a `<pre>` element with numbered lines. A specific line designated by the `highlightLine` field is visually highlighted to draw attention to the relevant code. By default, only the first three snippets are shown; a `Show N more` button toggles the display of the remaining snippets. The component also detects the source file language from its extension (Java, YAML, XML, JSON, or properties) for potential syntax highlighting.

Progress Tracker (`ProgressTrackerComponent`): This component accepts an `AnalysisJob` object and renders a multi-step progress indicator reflecting the pipeline's current state. The tracker displays a horizontal progress bar filled to the job's `progressPercentage`, the current status as a formatted badge, contextual details (current phase, current service being analyzed, and a services-completed counter), and a step indicator with six steps: Cloning, Detecting, Analyzing, Building Graph, Detecting Patterns, and Complete. Each step is classified as completed, active, or pending by comparing its position in a predefined order array against the current job status position. Completed steps display a checkmark icon, and the active step is visually highlighted. The component itself is purely presentational; the polling logic that updates the `AnalysisJob` input lives in the parent page component.

4.5 Deployment

The application is containerized using DOcker and orchestrated with DOcker Compose. Two configurations are provided: a development configuration for local development and a production configuration for deployment on a productive environment.

4.5.1 Docker Configuration

The backend is packaged using a multi-stage Dockerfile. The build stage uses an Eclipse Temurin JDK 25 Alpine image to compile the application with Maven. Docker's layer caching ensures dependency resolution is only re-executed when `pom.xml` changes by copying and resolving dependencies (`mvn dependency:go-offline`) before copying the source code. The runtime stage uses a minimal JRE 25 Alpine image, installs Git (required for repository cloning at runtime), creates a non-root user (`appuser`) for security and runs the application with container-aware JVM settings (`-XX:+UseContainerSupport`, `-XX:MaxRAMPercentage=75.0`) that allow the JVM to respect container memory limits. A health check endpoint (`/actuator/health`) is configured for container orchestration. Listing 4.10 shows the runtime stage.

Figure 4.10: Runtime stage of the multi-stage Dockerfile. The image uses a non-root user and container-aware JVM flags.

4.5.2 Docker Compose (Development)

The development Docker Compose configuration defines four services on a shared bridge network. The **PostgreSQL 16** (Alpine) container serves as the database, initialized via Flyway migration scripts [3] and exposed on port 5432 with a periodic health check (`pg_isready`). The **backend** container is built from the Dockerfile described above, connected to the database via environment variables, and mounts the DesigniteJava JAR as a read-only volume. It is exposed on port 8080 and depends on the database being healthy before starting. The **frontend** container is built from the sibling `dissertation_frontend` repository and served via Nginx on port 4200, with a startup dependency on the backend's health check. An optional **pgAdmin** container, enabled via the `tools` Docker Compose profile, provides a web-based database administration interface on port 5050 for development convenience. Named volumes (`postgres_data`, `workspace_data`) persist the database

[3]

and analysis workspace across container restarts, so that uploaded projects and analysis results are not lost during container restarts.

4.5.3 Docker Compose (Production)

The production configuration introduces several differences for security and resource management. Sensitive configuration, i.e. the database password and JWT signing secret, is injected via required environment variables (using Docker Compose's `${VAR:?error}` syntax) rather than hardcoded defaults, which prevents accidental deployment with insecure credentials. The database port is not exposed to the host, instead, only the frontend container exposes port 80 (configurable via `FRONTEND_PORT`), acting as the single entry point to the system. The frontend's Nginx configuration reverse-proxies API requests at the `/api` path to the backend container, which eliminates the need for CORS headers and ensures that all traffic enters through a single port. Resource limits are enforced via Docker's `deploy.resources.limits`: 1 GB for PostgreSQL, 2 GB for the backend, and 256 MB for the frontend. The backend's health check start period is increased to 60 seconds to accommodate longer JVM startup times in memory-constrained environments.

4.5.4 Configuration Management

The application configuration is externalized via environment variables with sensible defaults for local development, following established practices for cloud-native applications [**?**]. Spring Boot's property placeholder mechanism (`${VAR:default}`) in `application.yml` resolves environment variables at startup, with fallback values ensuring that the application runs out-of-the-box in a development environment without additional configuration. Table 4.6 lists the key configurable parameters.

4.5.5 Production Deployment Considerations

While the Docker Compose configurations described above are sufficient for single-server deployments, deploying the system to a publicly accessible production environment introduces additional concerns around hosting infrastructure, data access patterns, persistent storage, network security, and operational maintenance.

Hosting Infrastructure

The containerized architecture makes the application portable across any Docker capable hosting environment. A single virtual private server (VPS) from providers such as DigitalOcean, Hetzner or AWS EC2 with at least 4 GB of RAM and 2 vCPUs

Parameter	Default	Description
DB_HOST	localhost	Database hostname
DB_PORT	5432	Database port
DB_NAME	msadetector	Database name
JWT_SECRET	(dev key)	JWT signing secret
JWT_EXPIRATION	86400000 (24h)	JWT token lifetime in ms
WORKSPACE_DIR	/tmp/.../workspace	Analysis workspace path
DESIGNITE_JAR_PATH	/opt/.../DesigniteJava.jar	Path to DesigniteJava
CLONE_TIMEOUT	300	Git clone timeout (seconds)
ANALYSIS_TIMEOUT	1800	DesigniteJava timeout (seconds)
NANO_MAX_LOC	500	Nano service LOC threshold
NANO_MAX_ENDPOINTS	2	Nano service endpoint threshold
GOD_MIN_DOMAINS	3	God service God Class threshold
CHATTY_MIN_CALLS	5	Chatty service call count threshold

Table 4.6: Key configurable parameters and their default values.

can run all three containers (database, backend, frontend) via the production Docker Compose file. For higher availability, the containers could be dpeloyed to a managed container orchestration platform such as AWS ECS, Google Cloud Run or a self-hosted Kubernetes cluster, though the current architecture does not require horizontal scaling of the backend, as analysis jobs are inherently sequential per-project. A managed PostgreSQL service (e.g., AWS RDS, DigitalOcean Managed Databases) could replace the containerized database to offload backup management, automatic failover and connection pooling to the cloud provider.

Data Access and Code Snippet Storage

A key architectural decision affects how the application can be deployed: all analysis evidence, including source code snippets shown to the user, is read from the cloned/uploaded source files *during the analysis phase* and persisted as JSON text directly in the `detected_anti_patterns` database table (in the `code_snippets` column). When a user later reviews results, the code snippets are served from the database via the API, not read from the filesystem at request time. This design has two important consequences for deployment. First, the application does not require the analyses source code to remain on disk after the analysis is completed, as the workspace volume serves as a temporary staging area that could be periodically cleaned to reclaim disk space. Second, all data required to show results, including the dependency graph JSON, anti-pattern descriptions, affected service lists, remediation advice and code snippets with line numbers, resides entirely in PostgreSQL. This means that the database is the only stateful component that requires backup, and results remain accessible even if the workspace volume is lost.

The same pattern applies to the dependency graph: the graph structure (nodes and edges with metadata) is serialized to JSON during the result assembly phase and stored in the `dependency_graph_json` column of the `analysis_results` table. The frontend renders this JSON directly without querying the filesystem.

TLS Termination and Domain Configuration

The production Docker Compose configuration exposes only port 80 via the frontend's Nginx container. For a public deployment, HTTPS must be enabled to protect JWT tokens in transit. The recommended approach is to place a revese proxy such as Nginx, Caddy or Traefik in front of the Docker Compose stack to handle TLS termination with certificates obtained from Let's Encrypt. Caddy is particularly suitable for simple deployments, as it automatically provisions and renews TLS certificates with zero configuration beyond specifying the domain name. Alternatively, if deployed behind a cloud load balancer (e.g. AWS ALB, Cloudflare), TLS termination can be handled at the load balancer level, with unencrypted traffic forwared to the Nginx container over the internal network.

Backup and Data Persistence

The PostgreSQL database is the primary stateful component and requires regular backups. The named Docker volume (`postgres_data_prod`) persists data across container restarts, but does not protect against host-level failures. For production, automated `pg_dump` backups should be scheduled (e.g. via a cron job) and stored off-host in object storage such as AWS S3. The workspace volume (`workspace_data_prod`) stores cloned repositories and extracted ZIP archives. Its loss does not affect any existing results (as mentioned above). For Git-based projects, the workspace can always be reconstructed by re-cloning.

Resource Sizing and Scaling

The backend's memory consumption is dominated by the Spoon AST analysis of large microservice projects, which can require significant heap space when parsing thousands of Java files. The 2 GB memory limit defined in the production configuration is sufficient for most open-source projects, however, an increase to this limit might be required for very large enterprise codebases. The thread pool is configured with a maximum of 5 concurrent analysis workers. On a shared server, this should be reduced to match the available CPU cores to avoid contention. If multiple users sumbit analyses simultaneously, jobs are queued and processed in order. Horizontal scaling of the backed is possible but would require a shared filesystem or object storage for the workspace volume, as well as a mechanism to route job status

polling requests to the instance processing each job (e.g. sticky sessions or a shared job queue via Redis).

4.6 Summary

This chapter presented the design and implementation of the MSA Detector application. The high-level architecture consists of an Angular frontend, a Spring Boot backend, and a PostgreSQL database, all containerized with Docker. The data model comprises nine entities capturing users, projects, microservices, endpoints, dependencies, code smells, analysis jobs, results, and detected anti-patterns. The backend implements a RESTful API [**?**] with JWT authentication [**?**], asynchronous analysis pipeline orchestration, a pluggable anti-pattern detector architecture based on the Strategy design pattern [**?**], and a re-analysis diff feature inspired by SonarQube's `Clean As You Code` concept [**?**]. Deployment is managed through Docker Compose with separate development and production configurations. The chapter concluded with a discussion of production deployment considerations, noting that all analysis evidence is persisted in the database during analysis, making PostgreSQL the sole stateful component required for serving results. The next chapter evaluates the system's detection accuracy and performance on real-world open-source microservice projects.

Chapter 5

Results & Evaluation

This chapter presents the evaluation of the MSA Detector system described in the preceding chapters. It begins with the selection and characterization of the datasets used for evaluation, followed by the presentation of the detection results across each dataset. The results are then discussed in the context of the detection methodology and compared against related work. The chapter concludes with an analysis of the limitations of the approach and the threats to validity of the evaluation.

5.1 Dataset Selection

The evaluation of the MSA Detector requires open-source Java-based microservice projects of sufficient size and complexity to exercise the detection pipeline. This section describes the criteria used for dataset selection and characterizes the selected projects.

5.1.1 Selection Criteria

The following criteria were used to select evaluation datasets:

1. The project must be implemented in Java using the Spring Boot framework, as the detection pipeline relies on Spring-specific annotations and configuration conventions (see Chapter 3).
2. The project must contain at least three microservices, to enable meaningful detection of inter-service anti-patterns such as cyclic dependencies, chatty services and distributed monolith.
3. The project must be publicly available on GitHub or GitLab, to ensure reproducibility of the evaluation.

4. The project must use Maven or Gradle as a build system, as the microservice detection mechanism relies on the presence of `pom.xml`, `build.gradle` or `build.gradle.kts` files.

5.1.2 Selected Datasets

Based on the criteria described above, three open-source microservice projects were selected for evaluation. Table 5.1 summarizes the key characteristics of each project.

Project	Services	LOC	Description
Project A	–	–	–
Project B	–	–	–
Project C	–	–	–

Table 5.1: Summary of the open-source microservice projects selected for evaluation.

5.1.3 Dataset Characterization

Table 5.2 provides aggregate statistics across all selected datasets.

Metric	Value
Total microservices	–
LOC per service (range)	–
Endpoints per service (range)	–
Build systems	–
Communication mechanisms	–

Table 5.2: Aggregate statistics across all selected datasets.

5.2 Results

This section presents the detection results produced by the MSA Detector for each evaluated dataset. For each project, the detected anti-patterns, health score, code smell summary and dependency graph metrics are reported.

5.2.1 Overview of Detection Results

Table 5.3 presents the high-level detection results for each evaluated project. The health score is computed using the formula described in Chapter 3, Section 3.1.5, with letter grades assigned on the standard scale (A: 90–100, B: 80–89, C: 70–79, D: 60–69, F: below 60).

Project	Health Score	Grade	Anti-Patterns	Code Smells	Cycles	Coupling (C)
Project A	–	–	–	–	–	–
Project B	–	–	–	–	–	–
Project C	–	–	–	–	–	–

Table 5.3: Summary of detection results across all evaluated datasets.

5.2.2 Anti-Pattern Distribution

Table 5.4 presents the breakdown of detected anti-pattern instances by type across all evaluated projects.

Anti-Pattern	Project A	Project B	Project C	Total
Shared Database	–	–	–	–
Cyclic Dependency	–	–	–	–
Nano Service	–	–	–	–
God Service	–	–	–	–
Chatty Service	–	–	–	–
Hardcoded Endpoints	–	–	–	–
Distributed Monolith	–	–	–	–
API Versioning Absence	–	–	–	–

Table 5.4: Anti-pattern detection results by type across all evaluated datasets.

5.2.3 Per-Project Results

This section presents the detailed detection results for each evaluated project, including the detected microservices, the dependency graph, anti-pattern evidence, and the health score breakdown.

Project A

Project B

Project C

5.2.4 Health Score Analysis

5.2.5 Re-Analysis and Diff Results

5.3 Discussion

This section discusses the detection results in the context of the methodology, compares the findings against related work and examines the implications for practitioners.

5.3.1 Detection Accuracy

To assess the accuracy of the detection results, each detected anti-pattern was manually validated by inspecting the source code of the evaluated projects. The manual inspection involved examining the code snippets provided by the tool as evidence, verifying the reported dependency relationships, and checking configuration files for shared datasource declarations.

Anti-Pattern	TP	FP	FN	Precision	Recall
Shared Database	–	–	–	–	–
Cyclic Dependency	–	–	–	–	–
Nano Service	–	–	–	–	–
God Service	–	–	–	–	–
Chatty Service	–	–	–	–	–
Hardcoded Endpoints	–	–	–	–	–
Distributed Monolith	–	–	–	–	–
API Versioning Absence	–	–	–	–	–

Table 5.5: Detection accuracy by anti-pattern type. TP = true positives, FP = false positives, FN = false negatives.

5.3.2 Comparison with Related Tools

This section compares the detection results and capabilities of the MSA Detector against the related tools reviewed in Chapter 2, Section 2.4.5.

5.3.3 Threshold Sensitivity

5.3.4 Practical Implications

The results of this evaluation suggest several practical implications for development teams working with microservice architectures.

First, the MSA Detector can be integrated into existing development workflows to provide continuous architectural quality monitoring. The REST API exposed by the backend (see Chapter 4, Section 4.3.1) enables programmatic integration into CI/CD pipelines: a post-build step could submit the project for analysis and fail the pipeline if the health score falls below a configurable threshold or if critical anti-patterns are detected. This is analogous to how tools such as SonarQube enforce quality gates on code-level metrics [**?**].

Second, the composite health score provides a quantifiable metric that teams can track over time. The diff feature (see Chapter 4, Section 4.3.6) enables teams

to measure the concrete impact of refactoring efforts: when a team resolves a shared database anti-pattern, the subsequent re-analysis quantifies the health score improvement and confirms that no new anti-patterns were introduced.

Third, the evidence-based reporting approach — attaching source code snippets and affected service lists to each detected anti-pattern — reduces the effort required for developers to understand and act on the findings. Rather than receiving an abstract warning such as "cyclic dependency detected", the developer is shown the specific `@FeignClient` annotation or `RestTemplate` invocation that creates the cycle, along with the affected services and a remediation suggestion.

5.4 Limitations

This section acknowledges the limitations of the approach and the evaluation.

1. **Language restriction:** The detection pipeline is limited to Java-based microservice systems that use Spring Boot conventions. Projects written in other languages (e.g., Go, Python, Node.js) or using non-Spring frameworks are not supported. This restricts the tool's applicability to the subset of microservice systems built on the Spring ecosystem.

2. **Static analysis only:** The tool relies exclusively on static code analysis and does not incorporate runtime data (e.g., distributed tracing, actual call frequencies). As a result, statically detected dependencies may not reflect actual runtime behaviour, and anti-patterns that manifest only at runtime (e.g., performance-related chatty services) may not be detected accurately.

3. **Microservice detection heuristic:** The automated microservice detection mechanism identifies services by scanning for build files (`pom.xml`, `build.gradle`). While an exclusion list filters out test modules, example projects, and benchmarks, the heuristic may produce false positives for non-service submodules (e.g., shared libraries, utility modules) that contain build files but are not independently deployable services. Conversely, microservices that do not follow the standard directory structure may be missed.

4. **Dependency resolution limitations:** The inter-service dependency resolution relies on matching call targets (from `@FeignClient` annotations, `RestTemplate` invocations, and `WebClient` calls) to known service names using exact matching, port-based matching, and fuzzy substring matching. Dynamically constructed URLs, URLs assembled from multiple property placeholders, or URLs

resolved via external configuration services (e.g., Spring Cloud Config) cannot be resolved statically, potentially resulting in undetected dependencies.

5. **DesigniteJava integration constraints:** The code smell detection via DesigniteJava is subject to the tool's own limitations, including a configurable timeout for long-running analyses and potential incompatibility with newer Java language features. The Spoon compliance level is set to Java 17, meaning that Java syntax features introduced after version 17 (e.g., record patterns, unnamed variables) may not be parsed correctly in the analyzed projects.

6. **Dataset representativeness:** The evaluation was conducted on open-source sample and demonstration projects, which may not be representative of real enterprise microservice systems in terms of scale, complexity, development practices, and the presence of intentional or unintentional anti-patterns.

7. **Health score formula:** The linear deduction formula used for health score computation assigns fixed penalty weights to each severity level. These weights are not empirically validated and may not accurately reflect the relative impact of different anti-patterns in all project contexts. The equal weighting across categories may overemphasize or underemphasize certain quality dimensions depending on the project.

8. **Threshold generalizability:** The default thresholds for metrics-based anti-patterns (e.g., nano service LOC threshold of 500, chatty service call count of 5, god service God Class count of 3) were chosen based on values reported in the literature **[?]** and reasonable engineering judgment. These defaults may not be appropriate for all project contexts, and calibration is required for optimal results.

5.5 Threats to Validity

This section identifies and discusses the threats to the validity of the evaluation, organized by the standard classification of construct, internal, external and conclusion validity.

5.5.1 Construct Validity

Construct validity concerns whether the metrics and thresholds used in the detection pipeline accurately measure the intended concepts.

The nano service detection criterion (Equation 3.4) uses the conjunction of LOC and endpoint count as a proxy for "insufficient operational justification". However, a service with low LOC may still perform a critical function (e.g., an authentication service that delegates to an external identity provider), and a service with few endpoints may encapsulate complex internal logic. The conjunction of both metrics mitigates this risk but does not eliminate it entirely.

The god service detection criterion (Equation 3.5) uses God Class frequency as a proxy for "excessive responsibility concentration". While the presence of multiple God Classes is a strong indicator, a service may handle multiple responsibilities without triggering God Class detection if the responsibilities are distributed across many small classes. Conversely, a service with a single large domain model may trigger God Class detection without genuinely spanning multiple business domains.

The coupling coefficient (Equation 3.7) measures the ratio of actual to possible inter-service dependencies. While a high coupling coefficient does suggest tight coupling, it does not account for the directionality or nature of the dependencies. A system where all services depend on a single shared library service may have a high coupling coefficient without exhibiting the characteristics of a distributed monolith.

The composite health score formula (Equation 3.2) aggregates multiple quality dimensions into a single numeric value. Any such aggregation involves a loss of information, and the chosen penalty weights may not reflect the relative importance of different anti-patterns in all contexts. The category-based breakdown provided by `HealthScoreCalculator` partially addresses this by preserving per-category scores.

5.5.2 Internal Validity

Internal validity concerns whether the results are attributable to the detection methodology rather than to confounding factors such as implementation bugs or procedural errors.

The detection logic was manually reviewed and tested during development, but the absence of a comprehensive automated test suite (unit tests for each detector, integration tests for the full pipeline) means that subtle bugs in the detection logic may have affected the results without being caught. In particular, the dependency graph construction involves complex target resolution logic with cascading matching strategies; errors in this logic could produce incorrect edges in the dependency

graph, which would propagate to the cyclic dependency, chatty service, and distributed monolith detectors.

The detectors are executed sequentially and independently; each detector receives the same input (the project and its microservices) and produces results without knowledge of other detectors' output. This independence ensures that the order of detector execution does not influence outcomes. However, the detectors share data through the database: code smells saved by the DesigniteJava integration are queried by the God Service detector. If the code smell persistence step and the anti-pattern detection step are not correctly isolated transactionally, intermittent visibility issues could affect God Service detection accuracy.

The DesigniteJava results were parsed from CSV output files using a line-by-line parser that matches expected column positions. Changes in DesigniteJava's output format across versions could cause parsing errors, although the parser includes defensive checks for malformed lines.

5.5.3 External Validity

External validity concerns the generalizability of the results beyond the specific projects and configuration used in this evaluation.

The evaluation uses open-source sample and demonstration projects, which may differ from real enterprise systems in several ways. Enterprise systems tend to be larger in scale (hundreds of services, millions of LOC), employ more diverse communication mechanisms (gRPC, message queues, event streams), and evolve over years with multiple teams contributing. The anti-pattern distributions observed in the evaluated sample projects may not reflect those of enterprise systems.

The evaluated projects may have been designed as teaching examples and may intentionally contain (or intentionally lack) certain anti-patterns. For instance, a tutorial project may use a shared database for simplicity, which the detector would correctly flag, but this design choice may be intentional rather than accidental. Conversely, well-maintained sample projects may exhibit fewer anti-patterns than typical enterprise systems where architectural debt accumulates over time [**?**].

The results are specific to Java Spring Boot projects and do not generalize to microservice systems built with other languages or frameworks. Polyglot microservice systems — where services are implemented in different languages — are increas-

ingly common in practice, and the current tool cannot analyze non-Java services within such systems.

5.5.4 Conclusion Validity

Conclusion validity concerns the reliability and reproducibility of the conclusions drawn from the evaluation.

The manual validation of detection results was performed by a single person (the author of this dissertation), introducing a potential bias. A different reviewer might classify borderline cases differently, particularly for anti-patterns with subjective elements such as "nano service" (where the threshold between an appropriately small service and an excessively small one is context-dependent) or "god service" (where the boundary between a feature-rich service and an over-responsible one is debatable).

The detection results are deterministic for a given project and set of thresholds: running the tool twice on the same codebase with the same configuration produces identical results. However, as discussed in Section 5.3.3, different threshold configurations can lead to substantially different detection counts, which could affect the conclusions about anti-pattern prevalence and detection accuracy.

5.6 Summary

This chapter presented the evaluation of the MSA Detector system on a set of open-source Java-based microservice projects. The evaluation aimed to assess the tool's effectiveness in detecting architectural anti-patterns and providing actionable insights to improve microservice design quality.

Three projects were selected for evaluation, meeting the criteria of being Java Spring Boot applications with multiple microservices. The selected projects included a total of XX microservices and represented various domains such as e-commerce and finance. The detection pipeline was able to identify a range of anti-patterns across these projects, with varying implications for architectural quality.

The results showed that the MSA Detector could effectively identify common microservice anti-patterns such as shared databases, cyclic dependencies, and chatty services. The health scores provided a useful summary metric for assessing the overall architectural quality, with detailed breakdowns available for deeper analysis.

In terms of detection accuracy, the tool demonstrated high precision and recall rates for most anti-patterns, although some challenges were encountered with false positives and false negatives. The integration of code smell detection with architectural analysis was highlighted as a key strength of the tool, providing a more comprehensive assessment of microservice quality.

The evaluation also identified several limitations and threats to validity, including the restriction to Java Spring Boot projects, the reliance on static analysis, and the limited representativeness of the evaluation datasets. These factors may affect the generalizability of the results, and caution is advised in interpreting the findings as definitive measures of architectural quality.

Overall, the evaluation provided positive indications of the MSA Detector's capabilities, while also highlighting areas for improvement and further research. The next chapter will conclude this dissertation with a summary of contributions, reflections on the research process, and suggestions for future work in the area of microservice architecture analysis and improvement.

Chapter 6

Conclusions & Future Work

Concluzii ...